

SQLP 과목 III 튜닝 스타일 모의문제 · 3회차

SQL 자격검정 실전문제 3장 스타일 · 4지선다 · 총 20문항 · 원본 창작

1

Random 액세스와 Sequential 액세스, 그리고 읽어야 할 데이터량에 따른 I/O 효율에 대한 설명으로 가장 적절하지 않은 것은?

- ① 읽어야 할 데이터가 테이블의 상당 부분을 차지하는 대량 액세스에서도, 인덱스를 경유한 Random 액세스가 테이블 전체를 훑는 Sequential 액세스(Full Table Scan)보다 블록 I/O 효율이 높다.
- ② 인덱스 Random 액세스는 읽어야 할 행마다 인덱스에서 ROWID를 구해 테이블 블록을 한 블록씩 Single Block I/O로 찾아가므로, 결과 행 수가 늘수록 테이블 블록 I/O 횟수도 그에 비례해 증가하는 경향이 있다.
- ③ Sequential 액세스로 연속된 블록을 훑을 때는 한 번의 I/O Call로 여러 블록을 함께 읽는 Multiblock I/O를 쓸 수 있어, 같은 블록 수를 읽더라도 Random 액세스보다 I/O Call 횟수를 줄일 수 있다.
- ④ 소량의 행만 골라낼 때는 인덱스 Random 액세스가, 넓은 범위를 대량으로 읽을 때는 Full Table Scan의 Sequential 액세스가 대체로 유리해, 두 방식 사이에는 읽는 양에 따라 유·불리가 갈리는 손익분기점이 존재한다.

2

블록·익스텐트·세그먼트·테이블스페이스로 이어지는 데이터베이스 물리 저장 구조에 대한 설명으로 가장 적절한 것은?

- ① 테이블스페이스는 데이터가 물리적으로 담기는 최소 저장 단위이며, 하나의 테이블스페이스는 정확히 하나의 데이터파일에 대응된다.
- ② 고수위선(HWM)은 세그먼트에 할당된 마지막 익스텐트의 끝 경계를 가리키며, Full Table Scan은 HWM과 무관하게 세그먼트에 할당된 익스텐트 전체를 끝까지 읽는다.
- ③ 익스텐트는 논리적으로 연속된 블록들의 집합으로 공간 할당의 단위이며, 세그먼트에 공간이 부족하면 블록 하나씩이 아니라 익스텐트 단위로 새 공간이 추가된다.
- ④ Row Migration은 한 행이 하나의 블록에 다 담기지 못해 여러 블록에 나뉘어 저장되는 현상이고, Row Chaining은 갱신으로 행이 커져 원래 블록에 남지 못하고 다른 블록으로 옮겨 가는 현상이다.

도서관 야간 배치가 장서(도서 사본) 약 6만 권의 대출 현황을 대출 테이블에서 권당 한 건씩 조회한다. 원래는 청구기호를 문자열 리터럴로 이어붙여(WHERE 청구기호 = '813.7-...') SQL을 만들었는데, 이를 바인드 변수(WHERE 청구기호 = :b1)로 바꾼 뒤 두 번 트레이스를 수집했다. 아래 [바인드 전]·[바인드 후] 두 TKPROF에 대한 옳은 진단은?

아 래							
[바인드 전] 리터럴 SQL 6만 종							
Call	Count	CPU Time	Elapsed	Disk	Query	Current	Rows
Parse	60000	51.200	64.500	0	0	0	0
Execute	60000	9.100	10.200	0	120000	0	60000
Fetch	60000	3.800	14.100	900	240000	0	60000
Total	180000	64.100	88.800	900	360000	0	120000
Misses in library cache during parse: 60000							
[바인드 후] 바인드 변수 SQL 1종							
Call	Count	CPU Time	Elapsed	Disk	Query	Current	Rows
Parse	60000	6.400	7.100	0	0	0	0
Execute	60000	9.000	10.000	0	120000	0	60000
Fetch	60000	3.700	13.900	880	240000	0	60000
Total	180000	19.100	31.000	880	360000	0	120000
Misses in library cache during parse: 1							

- ① 바인드 전환으로 라이브러리 캐시 미스가 60,000→1로 떨어져 하드파싱이 100%→거의 0%가 됐고 Parse CPU가 51.2→6.4초(약 87.5% 감소)했다. 남은 6.4초는 커서를 찾아 공유하는 소프트파싱 비용이며, 이는 실행계획 품질과는 무관한 별개의 측이다.
- ② 바인드 후에도 Parse Count가 60,000 그대로이고 Parse CPU가 6.4초 남았으므로, 이 6.4초는 커서 공유에 실패해 여전히 하드파싱을 타는 잔량이다. Misses 1은 집계 시점의 표본일 뿐이니 session_cached_cursors를 키워 Parse Count 자체를 줄여야 6.4초가 사라진다.
- ③ 바인드 후 Parse CPU가 6.4초 남고 Fetch Elapsed도 13.9초로 여전한 것은, 리터럴마다 세우던 계획이 하나로 굳어 실행마다 다른 값에 맞는 계획을 못 세우기 때문이다. Execute Query 120,000·Fetch Query 240,000이 전후 동일한 것이 그 획일화의 증거이므로 근본 원인은 계획 품질 저하다.
- ④ Fetch Elapsed 14.1→13.9초, Disk 900→880, Fetch Query 240,000이 모두 그대로인데 Total Elapsed만 88.8→31.0초로 57.8초 줄었으니, 이 57.8초는 캐시 적중이 좋아져 물리 I/O 대기가 걸린 몫이다. 개선의 정체는 파싱이 아니며 처방은 버퍼 캐시 확대다.

4

전력사용량 집계 배치가 한 세션에서 수행되는 동안, 그 세션의 시간 모델과 대기 이벤트를 초 단위로 요약한 결과이다. 오라클의 응답 시간 분석(Response Time Analysis) 관점에서 이 배치 세션에 대한 해석으로 가장 적절한 것은?

아 래

[전력사용량 집계 배치 - 세션 시간 요약 (단위: 초)]

구분	항목	시간
Service Time	CPU used by session	420
Wait (비유휴)	db file sequential read	260
Wait (비유휴)	db file scattered read	90
Wait (비유휴)	log file sync	30
Wait (유휴)	SQL*Net message from client	150

비고: 'CPU used by session'은 이 세션이 소비한 서비스 시간이다.

- ① 응답 시간은 CPU used(420)에 모든 대기 시간(비유휴 380 + 유휴 150 = 530)을 더한 950초이며, 이 중 유휴 대기가 가장 큰 비중을 차지한다.
- ② 응답 시간(DB Time)은 서비스 시간(420)에 비유휴 대기(260+90+30=380)를 더한 800초이며, 유휴 대기인 SQL*Net message from client(150)는 응답 시간에 포함하지 않는다.
- ③ 이 세션은 비유휴 대기 시간(380)이 서비스 시간(420)보다 크므로, CPU가 아니라 I/O 대기가 응답 시간을 지배하는 대기 우세 세션이다.
- ④ 서비스 시간은 'CPU used by session'(420)과는 별개의 항목이므로, 두 값을 합한 840초를 이 세션의 서비스 시간으로 보아야 한다.

5

진료기록 테이블(약 7,000만 건)과 병원 테이블(약 3만 건)을 조인해 병원별 진료건수를 집계하는 SQL의 TKPROF 결과이다. 아래 트레이스에 대한 해석으로 가장 적절한 것은?

아 래

Call	Count	CPU Time	Elapsed Time	Disk	Query	Current	Rows
Parse	1	0.005	0.005	0	0	0	0
Execute	1	0.000	0.000	0	0	0	0
Fetch	401	8.400	61.200	104000	1600000	0	4000
Total	403	8.405	61.205	104000	1600000	0	4000
Rows	Row Source Operation						
4000	HASH GROUP BY (cr=1600000 pr=104000 pw=0 time=61150000 us)						
70000000	HASH JOIN (cr=1600000 pr=104000 pw=0 time=60000000 us)						
30000	TABLE ACCESS FULL 병원 (cr=90000 pr=6000 pw=0 time=1250000 us)						
70000000	TABLE ACCESS FULL 진료기록 (cr=1510000 pr=98000 pw=0 time=57800000 us)						

- ① BCHR이 93.5%로 높아 캐시는 넉넉한데 Elapsed가 61.2초인 것은, HASH JOIN이 7,000만 건을 해시 테이블로 올리며 해시 영역이 모자라 디스크로 스푼했기 때문이다. HASH JOIN의 time≈60.0초가 하위 진료기록 Full Scan의 57.8초보다 2.2초 큰 것이 그 스푼 뒀이므로, PGA를 키우면 61초가 사라진다.
- ② BCHR이 약 93.5%이고 HASH JOIN의 pw=0이라 캐시 부족도 해시 스푼도 없으니 I/O 쪽은 이미 정상이다. 남은 병목은 Fetch CPU 8.4초이며, 7,000만 건 조인과 4,000건 HASH GROUP BY를 한 프로세스가 혼자 처리한 탓이므로 병렬도(DOP)를 높여 CPU를 나눠 갖는 것이 최우선 처방이다.
- ③ Elapsed 61.2초와 CPU 8.4초의 약 52.8초 간극은 대기인데, 그 대기는 병원(3만 건)에 인덱스가 없어 TABLE ACCESS FULL(cr=90,000, pr=6,000)로 훑느라 HASH JOIN의 빌드 입력이 늦게 완성된 데서 비롯한다. 병원에 인덱스를 만들어 Full Scan을 없애면 61초가 해소된다.
- ④ 논리 I/O 160만 블록 중 10.4만(약 6.5%)을 디스크에서 읽어 BCHR은 약 93.5%이며, CPU 8.4초는 Elapsed 61.2초의 약 14%뿐이다. cr·pr·time이 모두 진료기록 Full Scan(cr=1,510,000, pr=98,000, time≈57.8초)에 집중되므로 병목은 7,000만 건 진료기록 테이블 액세스다.

6

학사성적 테이블(약 940만 건)에는 세 칼럼 결합 인덱스 성적_IX = (학과코드, 이수학기, 성적등급)이 있다. 학생명을 함께 얻기 위해 학생 테이블(기본키 학생_PK = 학번)과 조인하는 다음 SQL의 실행계획이다. 참고로 학과코드 = 'CSE'이면서 이수학기가 범위에 드는 성적 레코드는 약 26,000건이고, 그중 성적등급 = 'A0'인 것이 약 3,200건이다.

```
SELECT s.학번, s.이수학기, s.성적등급, st.학생명
FROM 학사성적 s, 학생 st
WHERE s.학과코드 = 'CSE'
AND s.이수학기 >= '2025-1학기'
AND s.이수학기 <= '2026-1학기'
AND s.성적등급 = 'A0'
AND st.학번 = s.학번;
```

이 실행계획에 대한 설명으로 가장 적절한 것은?

아 래					
Id	Pid	Operation	(Cost	Card	Bytes)
0	-	SELECT STATEMENT	(3718	3200	198400)
1	0	NESTED LOOPS	(3718	3200	198400)
2	1	TABLE ACCESS BY INDEX ROWID 학사성적	(518	3200	140800)
3	2	INDEX RANGE SCAN 성적_IX	(46	3200	)
4	1	TABLE ACCESS BY INDEX ROWID 학생	(1	1	18)
5	4	INDEX UNIQUE SCAN 학생_PK	(0	1	)

- ① 학과코드·이수학기·성적등급 세 조건이 모두 성적_IX의 구성 칼럼이므로 셋 다 인덱스 액세스 조건이 되어 함께 INDEX RANGE SCAN(Id 3)의 스캔 시작·종료 지점을 좁힌다. Id 3의 Cost 46과 Card 3,200이 세 조건으로 구간을 압축한 결과다.
- ② 이수학기가 BETWEEN 범위 조건이라 인덱스 정렬 연속성이 첫 칼럼에서부터 끊기므로, 선두 학과코드='CSE' 등치는 스캔 시작점을 고정하지 못하고 Id 3에서 필터로만 작동한다. Id 3의 Card 3,200은 리프를 폭넓게 훑은 뒤 남은 수다.
- ③ Id 3의 Cost 46과 Id 2의 Cost 518 사이 472의 차이는 스캔 구간에 든 약 26,000건을 ROWID로 테이블에서 읽은 비용이고, 성적등급은 Id 2 TABLE ACCESS BY INDEX ROWID에서 테이블 필터로 평가돼 3,200건이 남는다.
- ④ 학과코드 등치와 이수학기 범위까지가 인덱스 액세스 조건으로 스캔 구간을 결정하고, 성적등급은 성적_IX에 속하지만 범위 조건 뒤 칼럼이라 액세스 조건이 아니라 리프에서 걸러지는 인덱스 필터 조건이다.

7

자재입출고 테이블(약 1,200만 건, 세그먼트 약 150,000 블록)에서 자재입출고_IX 인덱스를 경유해 특정 기간의 입출고 내역을 조회한 SQL의 트레이스이다. 이 트레이스에 대한 옳은 진단은?

아래

Call	Count	CPU Time	Elapsed Time	Disk	Query	Current	Rows
Parse	1	0.006	0.007	0	0	0	0
Execute	1	0.000	0.000	0	0	0	0
Fetch	4801	6.100	41.500	62000	289200	0	480000
Total	4803	6.106	41.507	62000	289200	0	480000
Rows	Row Source Operation						
480000	TABLE ACCESS BY INDEX ROWID 자재입출고 (cr=289200 pr=62000 pw=0 time=41200000 us)						
480000	INDEX RANGE SCAN 자재입출고_IX (cr=1200 pr=300 pw=0 time=580000 us)						

- ① 인덱스 스캔은 $cr=1,200 \cdot time \approx 0.58$ 초로 이미 효율적이라 인덱스에는 문제가 없고, Parse·Execute가 0.006초뿐이라 41.5초는 전부 Fetch에서 났다. 병목은 48만 행을 Fetch 4,801회로 나눠 호출당 약 100행씩만 받은 데 있으니 Array Size를 키우면 해소된다.
- ② 테이블 랜덤 액세스가 ROWID 48만 건에 블록 I/O 28.8만을 써 CF 밀도가 0.6(테이블 블록 수 150,000의 약 48배)로 나뉘고, 읽은 행이 전체의 약 4%인데도 단일블록 랜덤 읽기 28.9만 > Full Scan 15만 블록이라 손익분기점을 넘었다. Full Scan(또는 인덱스 커버링)이 유리하다.
- ③ ROWID 48만 건당 테이블 블록 읽기가 $288,000/480,000 = 0.6$ 으로 1보다 작아 한 블록에서 여러 행을 얻은 셈이니 클러스터링 팩터는 좋은 편이다. 그렇다면 41.5초 대기는 테이블이 아니라 인덱스 쪽이며, $cr=1,200 \cdot pr=300$ 뿐인 좁은 리프에 48만 번 접근이 몰린 리프 블록 경합이 원인이다.
- ④ 물리 읽기 62,000이 논리 읽기 289,200의 약 21%여서 BCHR이 약 78.6%에 그치고, Elapsed 41.5초와 CPU 6.1초의 35.4초 간극이 그 21%를 디스크에서 읽느라 기다린 시간이다. 근본 원인은 버퍼 캐시 부족이므로 캐시를 키우면 이 SQL이 해결된다.

8

결합 인덱스에 조회 컬럼을 추가해 커버링(테이블 액세스 생략)을 노리는 설계 판단에 대한 설명으로 가장 적절하지 않은 것은?

- ① 쿼리가 필요로 하는 컬럼을 인덱스에 모두 포함시켜 인덱스만 읽고 처리하도록 만들면(Covering Index), 인덱스에서 얻은 ROWID로 테이블을 되짚는 Random 액세스를 제거해 조회 성능을 높일 수 있다.
- ② 인덱스에 조회 컬럼을 많이 포함할수록 테이블 액세스를 줄일 여지가 커지므로, 컬럼을 넉넉히 넣어 둔 인덱스일수록 모든 조회에서 더 효율적인 설계가 된다.
- ③ 커버링을 위해 컬럼을 더할지는 그 인덱스로 처리하는 조회의 빈도·이득과 그로 인한 DML 부하 증가를 견주어 판단해야 하며, 둘 사이에는 이득과 비용이 맞교환되는 손익분기점이 존재한다.
- ④ 인덱스에 컬럼을 추가하면 리프 엔트리의 길이가 늘어 인덱스 크기가 커지고, 해당 테이블에 DML이 일어날 때마다 갱신·정렬해야 할 인덱스 데이터가 많아져 저장·유지 부하가 증가한다.

9

결합 인덱스 (A, B)에서 선두 컬럼 A의 조건 형태(IN, 범위, 부재)에 따라 후행 컬럼 B가 스캔 범위 결정에 어떻게 기여하는지에 대한 설명으로 가장 적절하지 않은 것은?

- ① 선두 컬럼 A가 등치(=)로 한 점에 고정되면 그 구간 안에서 B의 정렬이 그대로 살아 있으므로, B의 범위 조건이 스캔 시작·끝점을 좁히는 액세스 조건으로 작동한다.
- ② 선두 컬럼 A가 IN 조건(A IN (v1, v2, ...))이면 옵티마이저는 IN-List Iterator로 각 값을 하나의 등치처럼 차례로 탐색하므로, 각 A 값 구간 안에서 후행 컬럼 B가 액세스 조건으로 스캔 범위를 좁힐 수 있다.
- ③ 선두 컬럼 A가 조건절에 아예 없으면, A의 서로 다른 값 개수가 적은 경우에 한해 Index Skip Scan으로 각 A 값 구간을 건너뛰며 B 조건을 탐색하는 예외 경로만 가능하다.
- ④ 선두 컬럼 A에 범위 조건(BETWEEN > 등)이 주어져도 옵티마이저가 이를 IN-List로 펼쳐 각 값을 등치처럼 반복 처리하므로, 후행 컬럼 B가 스캔 시작·끝점을 확정하는 액세스 조건으로 작동한다.

항공권 발권 테이블(약 2,300만 건)과 항공편 테이블(약 38만 건)을 조인하는 SQL의 실행계획이다. 발권에는 발권_IX = (발권지점, 발권일자)가, 항공편에는 기본키 (항공편ID, 출발일자)와 별도의 비유일 인덱스 항공편_IX = (항공편ID)가 있다. 발권지점 = 'GMP'이면서 발권일자 = DATE '2026-07-15'인 발권은 약 620건이다. 이 실행계획에 대한 설명으로 가장 적절한 것은?

아래

```
SELECT t.발권번호, t.발권일시, f.편명, f.출발공항
FROM 발권 t, 항공편 f
WHERE t.발권지점 = 'GMP'
      AND t.발권일자 = DATE '2026-07-15'
      AND f.항공편ID = t.항공편ID;
```

Id	Operation	Name
0	SELECT STATEMENT	
1	NESTED LOOPS	
2	TABLE ACCESS BY INDEX ROWID	발권
* 3	INDEX RANGE SCAN	발권_IX
4	TABLE ACCESS BY INDEX ROWID	항공편
* 5	INDEX RANGE SCAN	항공편_IX

- ① NESTED LOOPS(Id 1)의 비용은 인너로 반복 탐색되는 항공편(약 38만 건)의 크기에 좌우된다. Id 5 항공편_IX RANGE SCAN이 매번 38만 건 규모의 인덱스를 뒤지기 때문이니, 항공편을 줄이거나 항공편_IX를 (항공편ID, 출발일자)로 재구성해 탐색 폭을 좁히는 것이 핵심 처방이다.
- ② 인너 항공편(약 38만 건)을 항공편ID 순으로 먼저 한 번에 읽어 정렬·적재한 뒤 바깥 발권 620건과 병합하는 구조다. 그래서 Id 2 발권과 Id 4 항공편은 각각 한 번씩만 스캔되고, Id 5 항공편_IX RANGE SCAN도 플랜 전체에서 한 번만 수행된다.
- ③ 드라이빙(선행) 집합은 바깥 발권(Id 2)이고, 발권_IX로 걸러진 발권 약 620건 각각에 대해 항공편_IX INDEX RANGE SCAN(Id 5)이 반복 수행되어 인너 항공편이 조인된다. 인너 반복 횟수는 인너 크기가 아니라 드라이빙 발권 건수가 결정한다.
- ④ 이 플랜의 드라이빙은 Id 4·5로 접근하는 항공편이고 발권을 인너로 반복 탐색한다. 항공편 약 38만 건 각각에 대해 발권_IX RANGE SCAN(Id 3)이 재실행되므로, 조건에 걸리는 발권 건수가 많을수록 앞쪽 몇 건만 뽑고 멈추는 부분범위처리 이득이 커진다.

SELECT 절에 사용된 스칼라 서브쿼리(Scalar Subquery)의 수행과 캐싱에 대한 설명으로 가장 적절한 것은?

- ① 스칼라 서브쿼리 결과는 입력값과 무관하게 세션이 유지되는 동안 캐시에 남아 재사용되므로, 서브쿼리가 참조하는 테이블 크기나 바깥 쿼리 결과 행 수와 상관없이 실제 호출 횟수는 첫 행에서의 한 번으로 줄어든다. 같은 세션에서 뒤이어 수행되는 다른 SQL도 그 캐시를 그대로 물려받아 재실행이 생략된다.
- ② 스칼라 서브쿼리는 바깥 쿼리가 내보내는 행마다 매번 독립적으로 수행되며, 입력값이 같아도 캐시가 개입할 여지가 없어 실제 호출 횟수는 바깥 쿼리 결과 행 수와 그대로 같아진다. 따라서 호출을 줄이려면 캐싱에 기대지 말고 서브쿼리를 아우터 조인으로 직접 바꿔 쓰는 수밖에 없다.
- ③ 옵티마이저는 SELECT 절 스칼라 서브쿼리를 항상 조인 형태로 쿼리 변환해 실행하므로, 캐시 적중률이 높은 낮은 실행계획은 동일한 조인 플랜으로 수렴한다. 그래서 입력값 종류가 많아 캐시가 무력해지는 상황에서도 스칼라 서브쿼리 때문에 성능이 나빠지는 일은 생기지 않는다.
- ④ 스칼라 서브쿼리는 입력값을 키로 하는 해시 기반 캐시에 결과를 저장해, 입력값의 종류가 적으면 같은 값의 재호출을 캐시로 대체해 실행 횟수를 줄인다. 다만 캐시 크기에 한계가 있어 입력값 종류가 많으면 해시 충돌·항목 밀려남으로 그 효과가 떨어질 수 있다.

12

비용 기반 옵티마이저(CBO)가 통계정보로 예상 카디널리티와 비용을 산정하는 원리에 대한 설명으로 가장 적절하지 않은 것은?

- ① 선택도(selectivity)는 조건을 만족하는 행의 비율에 대한 추정치이며, 예상 카디널리티는 대체로 '전체 행 수 × 선택도'로 산정된다.
- ② 컬럼의 NDV가 클수록 그 컬럼에 만든 인덱스의 클러스터링 팩터도 함께 작아지므로, NDV가 큰 컬럼일수록 인덱스를 경유한 테이블 랜덤 액세스 비용이 낮게 산정된다.
- ③ 값 분포가 한쪽으로 치우친 컬럼에 히스토그램을 수집하면, NDV만으로 균일 분포를 가정할 때보다 특정 값의 선택도를 더 정확히 추정할 수 있다.
- ④ 등치(=) 조건이 걸린 컬럼의 NDV(서로 다른 값의 개수)가 클수록 그 조건의 선택도는 대략 $1/NDV$ 로 낮게 추정되어, 예상 카디널리티가 작아진다.

13

옵티마이저의 View Merging(뷰 병합)과 그 가능·불가 조건에 대한 설명으로 가장 적절한 것은?

- ① 인라인 뷰에 ROWNUM이 포함되면 뷰를 바깥 쿼리에 병합했을 때 결과가 달라질 수 있으므로, 옵티마이저는 그 뷰를 병합하지 않고 독립된 블록으로 수행한다.
- ② 뷰 정의에 GROUP BY가 있는 복합 뷰는 집계 결과를 먼저 만들어야 하므로 병합 대상에서 제외되며, 옵티마이저는 뷰를 개별 수행한 뒤에만 바깥 쿼리와 조인한다.
- ③ 복합 뷰가 병합되지 못하면 옵티마이저는 뷰를 그대로 수행한 결과를 바깥에서 필터링할 뿐, 바깥의 조건을 뷰 안으로 밀어 넣지는 못한다.
- ④ View Merging과 서브쿼리 Unnesting은 둘 다 블록을 하나로 합치는 변환이므로, ROWNUM 때문에 병합되지 못하는 뷰라면 그 뷰 안의 서브쿼리도 Unnesting되지 않는다.

14

카드승인 테이블의 승인상태 칼럼은 값 편중이 심해(대부분 'A'=승인, 극소수 'D'=거절) 히스토그램이 수집돼 있다. 아래 SQL을 바인드 변수 :b1로 반복 수행한 뒤 v\$sql에서 이 SQL의 자식 커서를 조회한 결과가 함께 주어졌다. 커서 공유와 실행계획 재사용에 대한 설명으로 가장 적절하지 않은 것은?

아래

```
SELECT 승인번호, 승인금액
FROM 카드승인
WHERE 승인상태 = :b1;
```

[v\$sql - 동일 SQL_ID(b7q2m9c4f0r3d)의 자식 커서]

CHILD_NUMBER	PLAN_HASH_VALUE	IS_BIND_SENSITIVE	IS_BIND_AWARE	ROWS_PROCESSED
0	1183726554	Y	N	12
1	3902514877	Y	Y	480000

- ① 두 행은 같은 SQL_ID(b7q2...)를 갖지만 CHILD_NUMBER가 0·1로 나뉘고 PLAN_HASH_VALUE가 서로 달라, 하나의 부모 커서 아래 실행계획이 다른 자식 커서 두 개가 존재한다.
- ② 승인상태에 히스토그램이 있어 커서가 bind sensitive(IS_BIND_SENSITIVE=Y)가 되었고, 값 편중에 따른 처리건수 차이를 감지해 bind aware 자식(1번, IS_BIND_AWARE=Y)이 추가로 생성되었다.
- ③ 바인드 변수를 썼으므로 SQL 텍스트가 동일하고, 따라서 바인드 값이 무엇이든 옵티마이저는 항상 자식 커서 0번 하나만 재사용해 같은 실행계획으로 처리한다.
- ④ 첫 실행 때 엿본(bind peeking) 값에 맞춰 계획이 정해지므로, 편중된 칼럼에서는 처음 들여다본 값이 이후 실행의 계획 품질을 좌우할 수 있다.

15

월마감 배치가 당월 택배운송장을 집계 테이블 운송장_집계에 아래 SQL로 적재한다. 운송장_집계는 NOLOGGING 속성이며, 배송지점 칼럼에 비유일 인덱스 운송장집계_IX가 달려 있다. 이 적재의 동작에 대한 설명으로 가장 적절하지 않은 것은?

아 래

```
-- 월마감: 당월 운송장을 집계 테이블로 적재
INSERT /*+ APPEND */ INTO 운송장_집계
SELECT 운송장번호, 배송지점, 접수일자, 운임
FROM 택배운송장
WHERE 접수일자 >= DATE '2026-06-01'
AND 접수일자 < DATE '2026-07-01';
COMMIT;
```

- ① /*+ APPEND */ 힌트는 직접 경로 삽입(Direct Path Insert)을 유도해 HWM 위의 새 블록에 데이터를 적재하며, 세그먼트 안의 기존 프리 블록이나 삭제로 생긴 빈 공간은 재사용하지 않는다.
- ② 대상 테이블이 NOLOGGING이고 직접 경로로 적재되면 데이터에 대한 redo가 최소화되지만, 그 구간은 미디어 복구로 되살릴 수 없으므로 적재 직후 백업을 받아야 안전하다.
- ③ 직접 경로 삽입은 대상 테이블에 배타적 잠금을 걸지 않으므로, 이 적재가 커밋되기 전에도 다른 세션이 운송장_집계에 일반 INSERT를 동시에 수행할 수 있다.
- ④ NOLOGGING은 데이터 블록 적재의 redo만 줄일 뿐이어서, 운송장집계_IX 인덱스는 적재와 함께 갱신·유지되며 그에 따른 redo와 유지 비용은 그대로 남는다.

16

통신요금 테이블은 청구연월을 NUMBER(6)(YYYYMM 정수) 칼럼으로 두고, 이 칼럼을 기준으로 월별 RANGE 파티셔닝되어 있다(아래 DDL). 각 선택지는 SELECT SUM(청구금액) FROM 통신요금 WHERE ...의 WHERE 절이다. Partition Pruning이 효과적으로 작동하여 필요한 파티션만 읽는 것으로 보기 가장 적절하지 않은 것은?

아 래

```
CREATE TABLE 통신요금 (
    청구id    NUMBER,
    회선번호  VARCHAR2(20),
    청구연월  NUMBER(6),    -- YYYYMM 정수 (예: 202603)
    청구금액  NUMBER
)
PARTITION BY RANGE (청구연월) (
    PARTITION pt_202601 VALUES LESS THAN (202602),
    PARTITION pt_202602 VALUES LESS THAN (202603),
    PARTITION pt_202603 VALUES LESS THAN (202604),
    PARTITION pt_202604 VALUES LESS THAN (202605),
    PARTITION pt_202605 VALUES LESS THAN (202606),
    PARTITION pt_202606 VALUES LESS THAN (202607),
    PARTITION pt_max   VALUES LESS THAN (MAXVALUE)
);
```

- ① WHERE TO_CHAR(청구연월) = '202603'
- ② WHERE 청구연월 BETWEEN 202601 AND 202602
- ③ WHERE 청구연월 >= 202605
- ④ WHERE 청구연월 = 202603

펀드거래 테이블(약 4.8억 건)과 펀드잔고 테이블(약 6,400만 건)은 모두 투자자ID로 32개 해시 파티션으로 나뉘어 있다. 두 테이블을 투자자ID로 조인하고 펀드유형별로 집계하는 야간 정산 배치 SQL의 병렬 실행계획이다. 이 플랜에서 두 테이블은 병렬 서버(Slave) 사이에 어떻게 분배되어 조인되는가? 가장 정확히 설명한 것은?

아래

Id	Operation	Name	Pstart	Pstop	TQ	IN-OUT
0	SELECT STATEMENT					
1	PX COORDINATOR					
2	PX SEND QC (RANDOM)	:TQ10001			Q1,01	P->S
3	HASH GROUP BY				Q1,01	PCWP
4	PX RECEIVE				Q1,01	PCWP
5	PX SEND HASH	:TQ10000			Q1,00	P->P
6	HASH GROUP BY				Q1,00	PCWP
7	PX PARTITION HASH ALL		1	32	Q1,00	PCWC
* 8	HASH JOIN				Q1,00	PCWP
9	TABLE ACCESS FULL	펀드거래	1	32	Q1,00	PCWP
10	TABLE ACCESS FULL	펀드잔고	1	32	Q1,00	PCWP

- ① 소형 펀드잔고(약 6,400만 건)를 Id 5 PX SEND HASH 경로로 병렬 서버 전체에 복제(broadcast)하고, 대형 펀드거래(약 4.8억 건)는 Id 9에서 Pstart 1~Pstop 32 그대로 각 서버가 자기 조각만 스캔해 재분배하지 않는 broadcast-none 분배다. 복제본이 각 서버 HASH JOIN(Id 8)의 빌드 입력이 되고 조인 키 재분배는 없다.
- ② 두 테이블을 조인 키 투자자ID의 해시로 양쪽 재분배(hash-hash)한 뒤 각 병렬 서버가 자기 버킷 범위의 투자자ID 행만 짝지어 HASH JOIN(Id 8)한다. 그 재분배가 Id 5 PX SEND HASH이고 받는 쪽이 Id 4 PX RECEIVE이며, Id 7 PX PARTITION HASH ALL의 1~32는 서버가 맡은 버킷 범위 표시다.
- ③ 두 테이블이 투자자ID로 동일하게 32개 해시 파티셔닝돼 있어 PX PARTITION HASH ALL(Id 7) 아래에서 각 병렬 서버가 대응하는 파티션 쌍만 HASH JOIN(Id 8)한다. 조인 입력 펀드거래·펀드잔고(Id 9·10)가 같은 Q1,00에서 PX SEND 없이 읽히므로 조인을 위한 재분배가 없는 파티션 와이즈 조인이다.
- ④ 대형 펀드거래(약 4.8억 건)를 Id 5 PX SEND HASH로 투자자ID 해시 재분배하고, 소형 펀드잔고(약 6,400만 건)는 병렬 서버 전체에 복제(broadcast)해 각 서버가 전체 사본을 들고 HASH JOIN(Id 8)하는 hash-broadcast 분배다. Id 7의 Pstart 1~Pstop 32는 복제 대상 파티션 범위를 가리킨다.

분석함수(윈도우 함수)의 정렬과 집합 연산자의 정렬·중복 제거에 대한 설명으로 가장 적절한 것은?

- ① PARTITION BY만 있고 ORDER BY가 없는 윈도우 함수는 행을 파티션별로 나눌 필요가 없어, 정렬이나 그룹핑 오퍼레이션 없이 입력 순서 그대로 처리된다.
- ② UNION은 두 집합을 합친 뒤 중복을 걸러 내려고 Sort Unique(또는 해시) 단계를 거치지만, UNION ALL은 중복 제거가 없어 그 정렬 단계 없이 두 집합을 이어 붙인다.
- ③ 집합 연산자는 중복을 제거하려고 정렬(Sort Unique)을 수반한다는 성질에 근거하면, UNION ALL도 집합 연산자이므로 Sort Unique로 중복을 제거한 뒤 결과를 반환한다.
- ④ INTERSECT는 두 집합의 공통 행만 남기는 연산이라 정렬이 필요 없고, 정렬을 통한 중복 제거는 오직 UNION에서만 발생한다.

19

SQL Server의 읽기 일관성과 격리수준(READ COMMITTED, RCSI, SNAPSHOT)이 Non-Repeatable Read를 다루는 방식에 대한 설명으로 가장 적절한 것은?

- ① 기본 READ COMMITTED는 읽은 데이터에 걸린 공유(S) 락을 트랜잭션이 끝날 때까지 유지하므로, 한 트랜잭션 안에서 같은 행을 다시 읽어도 값이 변하지 않는다.
- ② RCSI(READ_COMMITTED_SNAPSHOT)를 켜면 각 문장이 스냅샷을 읽으므로, 한 트랜잭션 안에서 여러 문장이 같은 행을 읽을 때 같은 값을 보장해 Non-Repeatable Read가 사라진다.
- ③ SNAPSHOT과 RCSI는 커밋된 데이터만 읽고 미커밋 데이터를 배제한다는 점이 같으므로, 두 격리수준의 Non-Repeatable Read 방지 수준도 서로 같다.
- ④ SNAPSHOT 격리수준은 트랜잭션 시작 시점의 버전(스냅샷)을 트랜잭션 내내 읽으므로, 같은 행을 두 번 읽어도 값이 바뀌지 않아 Non-Repeatable Read가 방지된다.

20

보험청구 테이블(object_id=154227)을 두고 두 세션이 동시에 작업 중이다. 세션 44는 정합성 점검을 위해 LOCK TABLE 보험청구 IN SHARE MODE를 수행했고, 세션 61은 같은 테이블에 UPDATE 보험청구 SET ...를 시도했다. 이때 v\$sqllock을 조회한 결과가 아래와 같다. 이 락 상태에 대한 해석으로 가장 적절하지 않은 것은?

아 래

[v\$sqllock - 보험청구 테이블 (ID1 = object_id = 154227)]

SID	TYPE	ID1	ID2	LMODE	REQUEST	BLOCK
44	TM	154227	0	4	0	1
61	TM	154227	0	0	3	0

* TM 모드 코드: 3 = RX(Row Exclusive) 4 = S(Share) 6 = X(Exclusive)

* S(4)와 RX(3)는 서로 호환되지 않는다.

- ① RX(모드 3)는 S(모드 4)와 호환되므로 세션 61은 44의 락과 무관하게 즉시 RX를 획득하며, BLOCK=1은 세션 61이 세션 44를 막고 있다는 표시다.
- ② 세션 61은 UPDATE를 위해 TM RX(모드 3)를 REQUEST했으나 LMODE가 0이라 아직 락을 획득하지 못하고 대기(wait) 상태다.
- ③ S(모드 4)와 RX(모드 3)는 호환되지 않으므로, 세션 61의 RX 요청은 세션 44가 S 락을 해제(커밋/롤백)할 때까지 대기한다.
- ④ 세션 44는 보험청구에 TM S(모드 4)를 LMODE로 보유하고 있고 BLOCK=1이므로, 다른 세션의 요청을 막고 있는 블로커(blocker) 쪽이다.

정답 및 해설

■ 정답 모아보기

1. ①	2. ③	3. ①	4. ②	5. ④
6. ④	7. ②	8. ②	9. ④	10. ③
11. ④	12. ②	13. ①	14. ③	15. ③
16. ①	17. ③	18. ②	19. ④	20. ①

문제 1. 정답 ①

두 액세스 방식은 읽는 양에 따라 유·불리가 반대로 갈립니다.

인덱스 Random 액세스: 얻어야 할 행마다 인덱스를 수직 탐색해 ROWID를 구하고, 그 ROWID가 가리키는 테이블 블록을 한 블록씩(Single Block I/O) 임의로 찾아갑니다. 결과 행이 늘수록 이 "한 건 → 한 블록" 왕복이 그만큼 반복되어 블록 I/O가 누적됩니다.

Full Table Scan의 Sequential 액세스: 세그먼트의 블록을 물리적 순서대로, 인접 블록을 묶어 Multiblock I/O로 훑습니다. 읽어야 할 양이 많을수록 한 번의 I/O Call로 여러 블록을 처리하는 이 방식이 효율적입니다.

그래서 소량이면 인덱스 Random, 대량이면 Full Scan Sequential이 유리하고, 그 사이에 유·불리가 뒤집히는 손익분기점이 존재합니다.

①은 이 관계를 정반대로 진술했습니다. 테이블의 상당 부분을 읽어야 하는 대량 액세스에서는 인덱스 Random 액세스가 오히려 불리합니다. 같은 테이블 블록을 여러 행이 공유하더라도 인덱스 경유 액세스는 논리적으로 매번 블록을 다시 방문하고, 흩어진 블록을 한 건씩 Single Block I/O로 집어야 하므로 I/O Call이 폭증합니다. "대량에서도 인덱스가 Full Scan보다 낫다"는 것은 소량에서만 성립하는 성질을 대량 구간까지 잘못 늘려 붙인 서술입니다.

②·③·④는 모두 옳습니다. Random 액세스의 건별 Single Block I/O 누적(②), Sequential 액세스의 Multiblock I/O로 인한 I/O Call 절감(③), 그리고 읽는 양에 따른 손익분기점의 존재(④)가 그 원리를 다른 각도에서 짚습니다.

선택지	판정	이유
①	×	대량 액세스에서는 Full Table Scan의 Sequential·Multiblock I/O가 유리합니다. 인덱스 Random 액세스가 대량에서도 낫다는 것은 소량에서만 맞는 성질을 뒤집어 늘린 서술입니다
②	○	인덱스 Random 액세스는 행마다 ROWID로 테이블 블록을 Single Block I/O로 찾아가므로 결과 행 수가 늘수록 블록 I/O가 비례해 증가합니다
③	○	연속 블록을 훑는 Sequential 액세스는 Multiblock I/O로 여러 블록을 한 번에 읽어 같은 블록 수라도 I/O Call 횟수를 줄일 수 있습니다
④	○	소량은 인덱스 Random, 대량은 Full Scan Sequential이 유리해 읽는 양에 따라 유·불리가 갈리는 손익분기점이 존재합니다

■ 이 문제의 핵심

1. Random 액세스 = 건별 Single Block I/O, Sequential 액세스 = 연속 블록 Multiblock I/O. 읽는 양이 유·불리를 가릅니다.
2. 인덱스 Random 액세스는 결과 행 수에 비례해 테이블 블록 I/O가 누적되므로, 읽을 행이 많아질수록 급격히 불리해집니다.
3. Full Table Scan은 Multiblock I/O로 대량을 한꺼번에 훑어, 넓은 범위에서 I/O Call을 크게 아깁니다.
4. 둘 사이에는 읽는 양에 따라 유·불리가 뒤집히는 손익분기점이 있습니다 — "인덱스가 언제나 낫다"는 발상이 함정입니다.
 - 한 줄 정리 소량은 인덱스 Random 액세스, 대량은 Full Scan Sequential 액세스가 유리하며, 대량 액세스에서도 인덱스가 낫다는 서술은 방향을 뒤집은 것입니다.

문제 2. 정답 ③

물리 저장 구조는 블록 C 익스텐트 C 세그먼트 C 테이블스페이스의 포함 관계로 쌓입니다.

블록 : I/O가 일어나는 최소 단위 (데이터가 실제로 담기는 그릇)
 익스텐트 : 연속된 블록들의 묶음 → 공간 할당의 단위
 세그먼트 : 테이블·인덱스 등 하나의 객체가 차지하는 익스텐트들의 집합

테이블스페이스 : 세그먼트들을 담는 논리 저장소 (하나 이상의 데이터파일로 구성)

③는 이 계층의 할당 단위를 정확히 짚었습니다. 세그먼트가 커져 공간이 모자라면 블록을 하나씩 붙이는 것이 아니라 익스텐트 단위로 한 묶음씩 새 공간을 확보합니다. 익스텐트가 "연속된 블록의 집합이자 할당 단위"라는 정의도 맞습니다. 나머지는 용어와 정의를 반사적으로 잘못 이어 붙인 함정입니다.

①: 물리적 최소 저장·I/O 단위는 블록이지 테이블스페이스가 아니며, 하나의 테이블스페이스는 여러 데이터파일로 구성될 수 있습니다. "테이블스페이스 ↔ 데이터파일 1:1"은 성립하지 않습니다.

②: 고수위선(HWM)은 세그먼트에서 한 번이라도 사용된 블록과 그 위쪽 미사용 블록을 가르는 경계입니다. Full Table Scan은 할당된 익스텐트 끝까지가 아니라 HWM까지의 블록만 읽습니다. HWM를 "할당 익스텐트의 끝"으로 본 것이 오류입니다.

④: Row Migration과 Row Chaining의 정의가 서로 뒤바뀌었습니다. 한 행이 한 블록에 다 못 담겨 여러 블록에 나뉘는 것이 Row Chaining, 갱신으로 커진 행이 원래 블록에 남지 못해 다른 블록으로 옮겨 가고 원 위치에 포인터만 남는 것이 Row Migration입니다.

선택지	판정	이유
①	×	물리적 최소 저장·I/O 단위는 블록이며, 하나의 테이블스페이스는 여러 데이터파일로 구성될 수 있습니다. "테이블스페이스 = 최소 단위·데이터파일 1:1"은 반사적 오연결입니다
②	×	HWM는 사용·미사용 블록의 경계이고 Full Table Scan은 HWM까지만 읽습니다. HWM를 "할당 익스텐트의 끝"으로 보고 익스텐트 전체를 읽는다는 서술은 틀립니다
③	○	익스텐트는 연속 블록의 집합이자 공간 할당 단위이며, 세그먼트 확장은 블록 단위가 아니라 익스텐트 단위로 이루어집니다. 계층과 할당 단위를 정확히 짚었습니다
④	×	정의가 뒤바뀌었습니다. 한 블록에 안 담겨 나뉘면 Row Chaining, 갱신으로 커져 다른 블록으로 옮겨 가면 Row Migration입니다

▪ 이 문제의 핵심

- 저장 계층은 블록 < 익스텐트 < 세그먼트 < 테이블스페이스. I/O·저장의 최소 단위는 테이블스페이스가 아니라 블록입니다.
- 공간 확장은 익스텐트 단위로 일어납니다. 블록 하나씩 붙는 것이 아닙니다.
- HWM는 사용/미사용 블록의 경계이고, Full Table Scan은 할당 익스텐트 끝이 아니라 HWM까지 읽습니다.
- Row Chaining = 한 행이 여러 블록에 나뉘, Row Migration = 커진 행이 다른 블록으로 이주(원 위치엔 포인터) — 정의를 반사적으로 맞바꾸지 않도록 주의합니다.

▪ 한 줄 정리 익스텐트는 연속 블록의 집합이자 공간 할당 단위이고 세그먼트 확장도 익스텐트 단위로 이루어지며, 최소 저장 단위·HWM·Row Chaining/Migration은 반사적으로 떠오르는 정의와 어긋나기 쉽습니다.

문제 3. 정답 ①

리터럴 SQL은 권마다 문자열이 달라 라이브러리 캐시에서 공유되지 못하고 매 실행이 하드파싱을 유발합니다. 바인드 변수로 바꾸면 SQL 문자열이 한 종으로 통일되어 하나의 공유 커서를 재사용(소프트파싱)합니다. 두 트레이스의 Misses와 Parse CPU가 이 전환을 그대로 보여줍니다.

하드파싱 비율 = 라이브러리 캐시 미스 ÷ Parse 횟수
 [바인드 전] 60,000 / 60,000 = 100% ← 파싱이 전부 하드파싱
 [바인드 후] 1 / 60,000 ≈ 0.0017% ← 커서 공유, 사실상 전부 소프트파싱

Parse CPU 감소 = (51.200 - 6.400) / 51.200 × 100 ≈ 87.5%

여기서 핵심은 Parse CPU가 0이 아니라 6.4초로 남았다는 점입니다. 바인드 후에도 Parse Count는 60,000 그대로이고, 각 파싱은 라이브러리 캐시에서 공유 커서를 찾아 붙는 소프트파싱 비용(캐시 latch·커서 탐색 CPU)을 치릅니다. 이 잔여 6.4초는 하드파싱이 남았다는 신호도, 실행계획이 나빠졌다는 신호도 아닙니다.

파싱 비용(하드/소프트) ———— 축 A: 커서를 만들/찾는 비용
 실행계획 품질(통계·카디널리티) ———— 축 B: 만들어진 계획이 좋은가

두 축은 독립: 미스가 60,000→1로 준 것은 축 A의 개선이고, Execute/Fetch의 Query(120,000·240,000)와 Rows(60,000)가 전후로 동일한 것은 계획(축 B)이 애초에 바뀌지 않았음을 보여줌.

Execute·Fetch의 Query·Disk·Rows가 전후로 거의 같다는 사실이 그 증거입니다. 데이터를 읽는 일(축 B)은 처음부터 같았고, 달라진 것은 오직 파싱 방식(축 A)뿐입니다. 따라서 ①이 옳은 진단입니다.

선택지	판정	이유
①	○	미스 60,000→1로 하드파싱 100%→약 0%, Parse CPU 51.2→6.4초(약 87.5% 감소)가 정확하고, 잔여 6.4초를 소프트파싱 비용으로, 계획 품질을 별개 축으로 옮겨 분리했습니다
②	×	Parse Count 60,000·Parse CPU 6.4초 인용은 맞지만 결론이 틀렸습니다. 60,000번 파싱 중 미스가 1이면 커서 생성이 단 한 번 일어난 것이라 하드파싱은 이미 사라졌고, 6.4초는 미스 없이 붙는 소프트파싱 CPU입니다. session_cached_cursors는 소프트파싱을 더 싸게 할 뿐 Parse Count 60,000 자체를 줄이지 못합니다
③	×	Parse CPU 6.4초와 Fetch Elapsed 13.9초 인용은 정확하나 귀속이 틀렸습니다. Execute Query 120,000·Fetch Query 240,000·Rows 60,000이 전후 동일한 것은 계획이 처음부터 바뀌지 않았다는 증거이지 품질이 떨어졌다는 증거가 아닙니다. 파싱 비용(축 A)과 계획 품질(축 B)을 인과로 묶은 별개 축 혼동입니다
④	×	Fetch Elapsed·Disk·Fetch Query가 그대로라는 관찰은 맞지만, Disk 20블록(900→880)·Fetch 0.2초로는 57.8초를 설명할 수 없습니다. 57.8초 중 57.4초는 Parse Elapsed 64.5→7.1초에서 났으므로, 개선의 정체는 물리 I/O가 아니라 파싱이고 버퍼 캐시 처방은 헛짚은 것입니다

■ 이 문제의 핵심

1. 하드파싱 비율 = **Misses in library cache during parse** ÷ **Parse Count**. 60,000/60,000=100%에서 1/60,000≈0%로 떨어진 것이 바인드 전환의 효과입니다.

2. 소프트파싱은 공짜가 아닙니다. 바인드 후에도 Parse Count는 60,000 그대로이고, 커서를 찾아 붙는 라이브러리 캐시 latch·탐색 CPU(6.4초)가 납니다. Parse CPU가 0이 아닌 것을 하드파싱 잔량이나 계획 문제로 오독하면 안 됩니다.

3. 파싱 비용(축 A)과 실행계획 품질(축 B)은 독립입니다. 미스 감소는 축 A 개선일 뿐, Execute·Fetch의 Query·Rows가 전후 동일해 계획은 처음부터 바뀌지 않았습니다.

4. 개선 폭의 정체는 Row Source가 아니라 Parse 라인(CPU 51.2→6.4, Elapsed 64.5→7.1)이며, Fetch·Disk가 정체 지점이 아님을 두 표를 나란히 놓아 확인합니다.

■ 한 줄 정리 바인드 전환으로 미스가 60,000→1(하드파싱 100%→0%)이 되고 Parse CPU가 51.2→6.4초로 줄어도, 남은 6.4초는 소프트파싱 비용이지 계획 품질 문제가 아니며 파싱 비용과 계획 품질은 별개의 축입니다.

문제 4. 정답 ②

응답 시간 분석의 기본 등식은 응답 시간 = 서비스 시간 + 대기 시간이고, 여기서 대기 시간은 비유휴(non-idle) 대기만 셉니다. 세션이 클라이언트의 다음 요청을 기다리는 **SQL*Net message from client** 같은 유휴(idle) 대기는 응답 시간(DB Time)에서 제외합니다. 또 하나 함정은 '서비스 시간'과 'CPU used by session'을 서로 다른 항목으로 오해하는 것인데, 비교가 명시하듯 둘은 같은 것을 다른 이름으로 부른 것입니다.

서비스 시간(Service Time) = CPU used by session = 420초 (동일 개념 - 이름만 다름)
비유휴 대기(Non-idle Wait) = 260 + 90 + 30 = 380초
유휴 대기(Idler Wait) = SQL*Net message from client = 150 = 응답시간에서 제외

응답 시간(DB Time) = 서비스 시간 + 비유휴 대기 = 420 + 380 = 800초
→ 서비스 비중 = 420 / 800 = 52.5%
→ 대기 비중 = 380 / 800 = 47.5% (서비스 시간이 대기 시간보다 크다)

계산하면 응답 시간은 800초이고 서비스 시간(420)이 비유휴 대기(380)보다 큼니다. 따라서 CPU가 아니라 I/O가 지배한다고 단정할 수 없습니다. ②가 등식·유휴 제외·수치를 모두 정확히 맞혔습니다.

선택지	판정	이유
①	×	유휴 대기(SQL*Net message from client 150)를 응답 시간에 포함시켰습니다. 유휴 대기는 DB Time에서 빼야 하므로 950초·유휴 최대 비중은 성립하지 않습니다
②	○	서비스 시간(420)+비유휴 대기(380)=800초, 유휴 대기 150초 제외. 응답 시간 분석 등식과 수치가 정확합니다
③	×	비유휴 대기 380초는 서비스 시간 420초보다 작습니다. 서비스 시간이 더 크므로 대기가 지배한다는 진술은 수치와 어긋납니다
④	×	'CPU used by session'이 곧 서비스 시간입니다. 같은 항목을 둘로 세어 840초로 이중 계산했습니다 — 서비스 시간은 420초입니다

■ 이 문제의 핵심

1. 응답 시간 = 서비스 시간(CPU) + 비유휴 대기 시간. 유휴 대기는 응답 시간(DB Time)에서 제외합니다.

2. 'CPU used by session' = 서비스 시간. 같은 것을 다른 이름으로 부른 것이라 둘을 더하면 이중 계산입니다.

3. **SQL*Net message from client**는 대표적 유헤 이벤트입니다 — 세션이 클라이언트를 기다리는 시간이라 병목이 아닙니다.
4. 대기 우세/서비스 우세 판정은 비유헤 대기와 서비스 시간의 크기 비교로 합니다. 여기선 서비스(420) > 비유헤 대기(380)입니다.
- 한 줄 정리 응답 시간은 서비스 시간(=CPU used 420)과 비유헤 대기(380)의 합인 800초이며, 유헤 대기 150초는 제외하고 서비스 시간이 대기보다 큼니다.

문제 5. 정답 ④

먼저 BCHR로 논리 대비 물리 읽기 비율을 확인합니다.

BCHR = ((Query + Current) - Disk) / (Query + Current) × 100
 = (1,600,000 + 0 - 104,000) / 1,600,000 × 100
 = 1,496,000 / 1,600,000 × 100 = 93.5%
 디스크 물리 읽기 비중 = 104,000 / 1,600,000 × 100 = 6.5%

논리 160만 블록 중 6.5%를 디스크에서 물리적으로 읽었습니다. 다음은 시간의 정체입니다.

CPU 비중 = 8.405 / 61.205 × 100 ≈ 13.7% → 나머지 약 86.3%가 대기
 Elapsed - CPU = 61.205 - 8.405 ≈ 52.8초 ← 일한 시간이 아니라 기다린 시간

응답시간의 대부분이 대기이고, 그 대기가 어디서 났는지는 Row Source가 지목합니다.

진료기록 cr 비중 = 1,510,000 / 1,600,000 × 100 ≈ 94.4% ← 블록 소비의 대부분
 진료기록 pr 비중 = 98,000 / 104,000 × 100 ≈ 94.2% ← 물리 읽기의 대부분
 진료기록 time = 57.8초 (전체 Elapsed 61.2초의 대부분)
 병원 cr 비중 = 90,000 / 1,600,000 × 100 ≈ 5.6% ← 소량

블록 소비(cr)·물리 읽기(pr)·소요 시간(time)이 세 지표 모두 진료기록 Full Scan 한 노드에 몰려 있고, HASH JOIN의 **pw=0**이라 해시 영역 스펠도 없습니다. 따라서 병목은 7,000만 건 진료기록 테이블의 Full Scan I/O이며, ④이 BCHR 해석과 병목 지목을 모두 옳게 했습니다.

선택지	판정	이유
①	×	HASH JOIN time≈60.0초와 진료기록 57.8초의 2.2초 차이는 맞게 읽었지만 결론이 틀렸습니다. HASH JOIN의 pw=0 은 해시 영역에서 디스크로 쓴 블록이 하나도 없다는 뜻이라 스펠 자체가 성립하지 않습니다. 대기의 정체는 진료기록 Full Scan의 물리 읽기(pr=98,000)이므로 PGA 확대는 헛짚은 처방입니다
②	×	BCHR 93.5%와 pw=0 판독은 옳으나 병목 지목이 틀렸습니다. CPU 8.405초는 Elapsed 61.205초의 약 13.7%뿐이고 나머지 약 86%가 대기이므로 CPU 바운드가 아닙니다. DOP를 올려도 진료기록 Full Scan이 읽는 pr=98,000 블록은 그대로 남습니다
③	×	52.8초 대기 계산과 병원의 cr=90,000·pr=6,000 인용은 정확하지만 귀속이 틀렸습니다. 병원의 time=1.25초는 61.2초의 2%에 불과하고 cr 비중도 5.6%뿐이며, 대기는 cr 94.4%·time 57.8초의 진료기록에서 났습니다. 3만 건 빌드 입력은 병목이 아닙니다
④	○	(1,600,000-104,000)/1,600,000 = 93.5%로 BCHR이 정확하고, cr(94.4%)·pr(94.2%)·time(57.8초)이 모두 진료기록 Full Scan에 집중되어 병목 지목이 맞습니다

■ 이 문제의 핵심

- BCHR = ((Query+Current)-Disk)/(Query+Current)×100. 여기서는 (1,600,000-104,000)/1,600,000 = 93.5%입니다.
 - CPU 비중(약 14%)이 낮으면 CPU 바운드가 아닙니다. Elapsed-CPU(약 52.8초)가 대기이고, 그 정체는 Row Source의 **pr·time**으로 특정 노드를 지목해야 드러납니다.
 - 병목 노드는 cr·pr·time이 가장 큰 곳 — 여기서는 진료기록 Full Scan(cr 94.4%, pr 94.2%, time 57.8초)입니다.
 - 높은 BCHR이 곧 효율은 아닙니다. 93.5%가 높아 보여도 논리 읽기 160만 블록 자체가 과다하면 캐시를 키워도 낭비는 남습니다.
 - pw=0**은 해시 영역 스펠이 없다는 신호 — PGA 부족을 원인으로 오귀속하지 않도록 반드시 확인합니다.
- 한 줄 정리 BCHR 93.5%와 CPU 14%까지 옳게 읽어도, cr·pr·time이 모두 진료기록 Full Scan에 몰리고 pw=0인 사실을 놓치면 병목을 CPU·병원 인덱스·PGA로 오귀속하게 됩니다.

문제 6. 정답 ④

성적_IX는 (학과코드, 이수학기, 성적등급) 순서입니다. WHERE의 세 조건을 인덱스 칼럼 순서에 대입해 어디까지가 스캔 구간을 정하는 액세스 조건인지를 따집니다.

학과코드 = 'CSE' 선두 칼럼 등치 → 액세스 조건 (스캔 시작점 고정)
 이수학기 BETWEEN ... 둘째 칼럼 범위 → 액세스 조건 (스캔 종료점 결정)
 성적등급 = 'A0' 셋째 칼럼 등치 → 범위 조건 '뒤'라 액세스 불가 → 인덱스 필터

인덱스 액세스 조건은 선두 칼럼부터 연속된 등치 + 첫 범위 칼럼까지만 될 수 있습니다. 둘째 칼럼 **이수학기**에서 범위(BETWEEN)가 걸리는 순간, 그 뒤 **성적등급**은 정렬 연속성이 끊겨 스캔 시작·종료 지점을 더 좁히지 못합니다. 대신 **성적등급 = 'A0'**은 리프 블록을 훑는 동안 각 엔트리에서 평가되는 인덱스 필터 조건이 됩니다.

스캔 구간 : 학과코드='CSE' AND 이수학기 2025-1~2026-1 → 리프 엔트리 약 26,000건 훑음
 인덱스 필터 : 그 26,000건에서 성적등급='A0'만 통과 → 약 3,200건 (Card)
 테이블 필터 : 없음 (WHERE의 학사성적 조건이 모두 인덱스 칼럼)

세 조건이 모두 **성적_IX**에 있지만, 스캔 범위를 정하는 것은 앞 두 조건뿐이고 성적등급은 리프에서 거르기만 합니다. Id 3(INDEX RANGE SCAN)과 Id 2(TABLE ACCESS)의 Card가 둘 다 3,200으로 같다는 점이, 성적등급이 이미 인덱스 단계에서 처리돼 테이블 단계에서는 추가로 줄어든 것이 없음을 보여 줍니다. 따라서 ④가 이 구조를 정확히 서술합니다.

선택지	판정	이유
①	×	세 조건이 모두 성적_IX 구성 칼럼이라는 사실과 Id 3의 Cost 46·Card 3,200 인용은 맞지만 결론이 틀렸습니다. 성적등급은 범위 조건(이수학기) 뒤 칼럼이라 스캔 시작·종료를 좁히지 못합니다. 셋 다 액세스 조건이었다면 리프를 26,000건이 아니라 3,200건만 훑었을 것이므로, 필터를 액세스로 뭉뚱그린 진술입니다
②	×	이수학기가 BETWEEN 범위라는 관찰은 맞지만 순서를 거꾸로 적용했습니다. 정렬 연속성은 범위 칼럼 '뒤'에서 끊기지 그 '앞'에서 끊기지 않으므로, 선두 등치 학과코드='CSE'는 그대로 스캔 시작점을 고정하는 액세스 조건입니다. 뒤 칼럼이 범위라고 선두 등치가 필터로 강등되지는 않습니다
③	×	Id 3(46)과 Id 2(518)의 Cost 차이가 테이블 랜덤 액세스 비용인 것은 맞지만, 그 대상은 26,000건이 아니라 인덱스 필터를 통과한 3,200건입니다. 성적등급은 성적_IX 구성 칼럼(학과코드, 이수학기, 성적등급)에 들어 있어 리프에서 걸러지는 인덱스 필터이며, 테이블 필터였다면 Id 2의 Card가 Id 3보다 작아야 하는데 둘 다 3,200입니다
④	○	학과코드(등치)·이수학기(범위)까지가 액세스 조건으로 스캔 구간을 정하고, 성적등급은 인덱스에 있으나 범위 뒤라 리프 필터로 평가됩니다 — Id 3·Id 2 Card가 3,200으로 같은 흐름과 일치합니다

■ 이 문제의 핵심

1. 인덱스 액세스 조건은 선두 칼럼부터 연속 등치 + 첫 범위 칼럼까지입니다. 여기서는 학과코드(=)·이수학기(범위)가 스캔 구간을 정합니다.
2. 범위 조건 뒤 칼럼은 인덱스에 있어도 액세스 조건이 못 됩니다. 성적등급은 리프를 훑으며 거르는 인덱스 필터 조건입니다.
3. 인덱스 필터 ≠ 테이블 필터. 인덱스에 있으면 리프에서(성적등급), 인덱스에 없는 컬럼 조건이면 테이블 액세스 후 필터로 걸러집니다.
4. Card 흐름이 증거. Id 3(INDEX RANGE SCAN)과 Id 2(TABLE ACCESS)의 Card가 3,200으로 같으면 테이블 단계 감소가 없다는 뜻 — 성적등급이 이미 인덱스에서 처리됐습니다.

■ 한 줄 정리 학과코드 등치와 이수학기 범위까지가 스캔 구간을 정하는 인덱스 액세스 조건이고, 그 뒤 성적등급은 인덱스에 있어도 리프에서 거르는 인덱스 필터 조건이라 Id 3·Id 2 Card가 3,200으로 같습니다.

문제 7. 정답 ②

트레이스의 세 숫자를 나란히 놓습니다.

480,000 ← 인덱스에서 얻은 ROWID 수 (INDEX RANGE SCAN 의 Rows)
 289,200 ← 테이블 액세스 누적 블록 I/O (TABLE ACCESS 의 cr)
 1,200 ← 인덱스 스캔에 든 블록 (INDEX RANGE SCAN 의 cr)

클러스터링 팩터(CF)는 인덱스 순서로 테이블을 읽을 때 발생하는 블록 I/O 수로, 이론상 최솟값은 테이블 블록 수(150,000), 최댓값은 테이블 행 수(12,000,000)입니다.

테이블 랜덤 액세스 블록 = 289,200 - 1,200 = 288,000
 CF 밀도 ≈ 테이블 랜덤 블록 ÷ ROWID 수 = 288,000 / 480,000 = 0.6 (ROWID당 블록)
 CF 추정 ≈ 0.6 × 12,000,000 = 7,200,000
 → 테이블 블록 150,000의 약 48배 (최솟값의 48배)
 → 최솟값 밀도(150,000/12,000,000 = 0.0125)의 48배로 분명히 나쁜 쪽

이 정도 CF에서 인덱스 경로의 실제 비용을 손익분기점 관점으로 따집니다.

읽은 행 비율 = 480,000 / 12,000,000 = 4.0% ← 흔히 '인덱스가 유리'로 보이는 범위
 인덱스 경로 블록 I/O = 288,000(테이블 랜덤, 대부분 단일블록) + 1,200(인덱스) = 289,200
 Full Scan 예상 블록 = 150,000 (multiblock 읽기)
 → 289,200 > 150,000: 이 범위에선 손익분기점을 넘어 Full Scan이 유리

읽은 행이 4%에 불과해도, 나쁜 CF 탓에 단일블록 랜덤 읽기 28.9만이 Full Scan 15만 블록을 크게 웃돕니다. Elapsed 41.5초 대비 CPU 6.1초의 약 35.4초 간극은 물리 읽기(Disk 62,000) 대기이며, 그 뿌리는 테이블 랜덤 액세스(pr=62,000)입니다. 처방은 인덱스 재정렬이 아니라 Full Scan 전환 또는 필요한 컬럼을 인덱스에 얹은 커버링(테이블 액세스 제거)입니다. 따라서 ②이 옳은 진단입니다.

선택지	판정	이유
①	×	인덱스 cr=1,200·time 0.58초, 41.5초가 Fetch에서 났다는 관찰, 호출당 약 100행(480,000÷4,801)까지 모두 맞지만 처방이 틀렸습니다. Array Size는 네트워크 왕복 횟수만 줄일 뿐, 41.5초의 정체인 테이블 랜덤 블록 288,000과 물리 읽기 62,000은 그대로 남습니다
②	○	테이블 랜덤 블록 288,000 / ROWID 480,000 = 0.6으로 CF는 테이블 블록의 약 48배(나쁨)이고, 랜덤 읽기 289,200 > Full Scan 150,000이라 손익분기점을 넘겨 Full Scan이 유리합니다
③	×	288,000/480,000 = 0.6이라는 산술은 맞지만 비교 기준이 틀렸습니다. 견줄 대상은 1.0이 아니라 최솟값 밀도(150,000/12,000,000 ≈ 0.0125)이고 0.6은 그 48배로 나쁜 쪽입니다. 41.5초도 리프 경합이 아니라 테이블 랜덤 액세스의 물리 읽기(전체 pr 62,000 중 인덱스 pr 300을 뺀 61,700) 대기입니다
④	×	(289,200-62,000)/289,200 ≈ 78.6%와 35.4초 간극 계산은 맞지만, 낭비의 정체는 캐시가 아니라 나쁜 CF가 만든 논리 읽기 289,200 자체입니다. 캐시를 키워 물리 읽기를 0으로 만들어도 28.8만 블록 랜덤 액세스와 그 CPU는 그대로 남고, Full Scan 15만 블록보다 여전히 많습니다

▪ 이 문제의 핵심

1. CF 밀도 ≈ 테이블 랜덤 블록(cr) ÷ ROWID 수. 0.6은 최솟값 밀도(약 0.0125)의 48배로, 최악(1.0)은 아니어도 분명히 나쁜 값입니다.
2. 테이블 랜덤 블록 = TABLE ACCESS cr - INDEX cr = 289,200 - 1,200 = 288,000. 병목은 인덱스가 아니라 이 랜덤 액세스입니다.
3. 손익분기점: 읽은 행이 4%라도 랜덤 단일블록 읽기(28.9만)가 Full Scan(15만 블록, multiblock)을 넘으면 Full Scan이 유리합니다.
4. 높은 BCHR이 곧 효율은 아닙니다. 논리 읽기 28.9만 블록 자체가 과다하면 캐시로는 해결되지 않습니다.
5. 나쁜 CF의 처방은 인덱스 재정렬이 아니라 Full Scan 전환 또는 커버링 인덱스로 테이블 액세스를 없애는 것입니다.

▪ 한 줄 정리 ROWID 48만에 테이블 블록 28.8만이면 CF 밀도는 0.6(테이블 블록의 약 48배)으로 나쁘고, 4% 범위라도 랜덤 28.9만 > Full Scan 15만이라 손익분기점을 넘겨 Full Scan이 유리합니다.

문제 8. 정답 ②

컬럼을 인덱스에 더하는 커버링 설계는 특정 조회에서 테이블 Random 액세스를 없애 주는 이득이 있습니다. 하지만 이는 그 컬럼들을 실제로 필요로 하는 조회에 한정된 이득입니다.

이득 : 이 인덱스로 처리되고, 추가한 컬럼을 SELECT/조건에 쓰는 조회 → 테이블 액세스 생략
 비용 : 리프 엔트리 길이 ↑ → 인덱스 크기 ↑, 블록당 엔트리 수 ↓
 + 그 테이블의 모든 DML마다 인덱스 갱신 부하 ↑

②는 이 부분적 이득을 전체로 확대한 함정입니다. "많이 넣을수록 모든 조회에서 더 효율적"이라는 결론은 두 지점에서 무너집니다.

그 컬럼을 안 쓰는 조회에는 이득이 없습니다. 오히려 엔트리가 길어져 같은 데이터를 담는 데 리프 블록이 더 필요해지고, 인덱스 스캔 시 읽을 블록이 늘어 손해만 봅니다.

DML 부하는 조회 종류와 무관하게 늘어납니다. 테이블에 행이 바뀔 때마다 커진 인덱스를 함께 갱신해야 하므로, 컬럼을 넉넉히 넣을수록 INSERT/UPDATE/DELETE가 그만큼 무거워집니다.

즉 컬럼 추가는 "많이 넣을수록 좋은" 일방적 이득이 아니라, ③이 말하는 이득과 비용의 손익분기점을 따져 필요한 만큼만 더하는 판단입니다.

①·④·③은 옳습니다. 커버링으로 Random 액세스를 제거하는 이득(①), 컬럼 추가에 따르는 인덱스 크기·DML 부하 증가(④), 그리고 이 둘을 건주는 손익분기점의 존재(③)가 커버링 설계의 세 축을 그대로 짚습니다.

선택지	판정	이유
①	○	필요한 컬럼을 인덱스에 모두 담아 인덱스만 읽고 처리하면 ROWID로 테이블을 되짚는 Random 액세스를 없애 조회 성능을 높일 수 있습니다
②	×	커버링 이득은 그 컬럼을 쓰는 조회에 한정됩니다. "많이 넣을수록 모든 조회에서 더 효율적"은 부분 이득을 전체로 확대한 서술로, 안 쓰는 조회의 스캔 손해와 DML 부하 증가를 무시합니다
③	○	컬럼 추가 여부는 조회 이득과 DML 부하 증가를 견주는 문제이며, 이득과 비용이 맞교환되는 손익분기점이 존재합니다
④	○	컬럼을 추가하면 엔트리가 길어져 인덱스가 커지고, 테이블 DML마다 갱신할 인덱스 데이터가 늘어 저장·유지 부하가 증가합니다

■ 이 문제의 핵심

1. 커버링 인덱스는 필요한 컬럼을 인덱스에 담아 테이블 Random 액세스를 제거하는 기법입니다.
2. 그 이득은 해당 컬럼을 실제로 쓰는 조회에 한정됩니다 — 쓰지 않는 조회에는 스캔 블록만 늘어 손해입니다.
3. 컬럼 추가는 인덱스 크기와 DML 부하를 조회 종류와 무관하게 키웁니다.
4. 그래서 설계는 "많이 넣기"가 아니라 이득과 비용의 손익분기점을 따져 필요한 만큼만 더하는 판단입니다.

■ 한 줄 정리 커버링을 위한 컬럼 추가는 그 컬럼을 쓰는 조회에 한정된 이득이자 DML 부하와의 손익분기점 문제이지, 많이 넣을수록 모든 조회가 빨라지는 설계가 아닙니다.

문제 9. 정답 ④

후행 컬럼 **B**가 스캔 범위를 좁히는 액세스 조건이 될 수 있는지는, 선두 컬럼 **A**가 하나의 값(점)으로 고정되어 그 안에서 **B**의 정렬이 살아 있느냐에 달려 있습니다.

A = v : A가 한 점 → 그 안에서 B 정렬 유지 → B가 액세스 조건 (범위 좁힘)

A IN (v1, v2) : 각 값이 등치처럼 하나의 점 → 값마다 그 안에서 B 정렬 유지 → B가 액세스 조건

A BETWEEN ... : A가 연속 구간에 열림 → 그 안에서 B가 모이지 않음 → B는 필터 조건

IN 조건은 유한한 개별 값의 나열이라 IN-List Iterator가 각 값을 하나의 등치점으로 펼쳐 반복 탐색할 수 있습니다. 그래서 IN에서는 값마다 **B**가 액세스 조건으로 작동합니다.

④은 이 IN-List의 경계를 범위 조건까지 잘못 늘렸습니다. **BETWEEN** > 같은 범위 조건은 시작·끝 사이의 연속된 무한한 값을 뜻하므로 개별 값으로 펼쳐 IN-List로 만들 수 없습니다. 선두 **A**가 범위로 열리면 그 구간 안에서 **B**는 값마다 흩어져 있어, **B**는 스캔 시작·끝점을 확정하지 못하고 필터 조건으로만 걸러 냅니다. "범위 조건도 IN-List로 펼쳐져 **B**가 액세스 조건이 된다"는 것은 IN에서만 성립하는 성질을 범위 상황에 옮겨 붙인 경계 오해입니다.

②·③·①는 옳습니다. IN 조건의 IN-List Iterator 처리(②), 선두 컬럼 부재 시 값 종류가 적을 때만 성립하는 Skip Scan 예외(③), 선두 등치 고정 시 후행 범위의 액세스 조건화(①)가 모두 정렬 원리에 맞습니다.

선택지	판정	이유
①	○	선두 A 가 등치로 한 점 고정되면 그 안에서 B 의 정렬이 살아 있어 B 의 범위 조건이 스캔 시작·끝점을 좁히는 액세스 조건이 됩니다
②	○	IN 은 개별 값의 나열이라 IN-List Iterator가 각 값을 등치점으로 반복 탐색하므로, 값마다 후행 B 가 액세스 조건으로 범위를 좁힐 수 있습니다
③	○	선두 컬럼이 조건에 없을 때는 값 종류가 적은 경우에 한해 Skip Scan으로 각 구간을 건너뛰며 후행 조건을 탐색하는 예외 경로만 가능합니다
④	×	범위 조건은 연속 값이라 IN-List로 펼칠 수 없습니다. 선두가 범위로 열리면 후행 B 는 필터로만 작동하며, IN에서만 성립하는 성질을 범위에 잘못 확장한 경계 오해입니다

■ 이 문제의 핵심

1. 후행 컬럼이 액세스 조건이 되려면 선두 컬럼이 하나의 점으로 고정되어 그 안에서 후행의 정렬이 살아야 합니다.
2. IN 조건은 개별 값의 나열이라 IN-List Iterator가 각 값을 등치점으로 펼쳐, 값마다 후행이 액세스 조건이 됩니다.
3. 범위 조건은 연속 값이라 IN-List로 펼칠 수 없습니다 — 선두가 범위면 후행은 필터로만 작동합니다. 이것이 이 문제의 경계입니다.
4. 선두 컬럼이 아예 없으면 값 종류가 적을 때의 Skip Scan만 예외 경로이며, 범위 조건의 비효율을 자동으로 풀어 주는 장치는 아닙니다.

- 한 줄 정리 IN 조건은 개별 값으로 펼쳐져 후행이 액세스 조건이 되지만, 연속 값인 범위 조건은 IN-List로 펼쳐지지 않아 후행이 필터로만 작동합니다.

문제 10. 정답 ③

NESTED LOOPS(Id 1) 아래 두 자식의 위치와 접근 경로를 읽습니다.

```

Id 1 NESTED LOOPS
Id 2 TABLE ACCESS BY INDEX ROWID 발권      ← 먼저 오는 자식 = 드라이빙(선행)
Id 3 INDEX RANGE SCAN 발권_IX              ← 드라이빙 집합 축소 (620건)
Id 4 TABLE ACCESS BY INDEX ROWID 항공편    ← 나중 자식 = 인너(후행), 반복 수행
Id 5 INDEX RANGE SCAN 항공편_IX            ← 인너 접근 경로 (드라이빙 건마다 재실행)

```

NESTED LOOPS에서 먼저 나열된 자식이 드라이빙(선행) 집합입니다. 여기서는 발권이 **발권_IX**(Id 3)로 **발권지점='GMP' AND 발권일자=...** 620건까지 좁혀지고, 그 각 행마다 인너 항공편을 **항공편_IX** INDEX RANGE SCAN(Id 5)으로 반복 탐색합니다. 즉 인너 접근 횟수는 드라이빙 발권 건수(620)만큼이며, 인너 항공편이 38만 건이든 380만 건이든 반복 횟수 자체는 드라이빙이 결정합니다.

인너 반복 횟수 = 드라이빙(발권) 건수 = 약 620회
 ≠ 인너(항공편) 테이블 크기 38만

그래서 이 조인의 부담을 줄이는 정공법은 인너 테이블 크기를 줄이는 것이 아니라 드라이빙 집합(발권 620건)을 더 좁히거나 인너 탐색을 값싸게(적절한 인덱스로) 만드는 것입니다. NL은 드라이빙 앞쪽부터 조인해 내려가므로, 상위 일부만 필요할 때 전체를 완성하지 않고 첫 행을 빠르게 내는 부분범위처리에도 유리합니다. ③가 드라이빙 지목과 반복 구조를 모두 정확히 서술합니다.

선택지	판정	이유
①	×	인너 항공편이 약 38만 건이라는 수치와 Id 5가 인너 접근 경로라는 판독은 맞지만 병목 귀속이 틀렸습니다. NL의 인너 접근 횟수는 인너 크기가 아니라 드라이빙 발권 620건이 결정하고, 각 탐색은 항공편_IX 를 타고 항공편ID 하나로 들어가므로 38만 건을 훑지 않습니다 — 줄여야 할 것은 드라이빙 집합입니다
②	×	한쪽을 정렬·적재한 뒤 병합하는 것은 소트 머지 조인의 서술입니다. 이 플랜엔 SORT JOIN도 HASH JOIN 노드도 없고 Id 1이 NESTED LOOPS이므로, Id 5 항공편_IX RANGE SCAN은 한 번이 아니라 드라이빙 발권 620건마다 재실행됩니다
③	○	먼저 나온 자식 발권(Id 2)이 드라이빙이고, 620건 각각에 항공편_IX RANGE SCAN(Id 5)이 반복됩니다 — 반복 횟수는 드라이빙 건수가 결정합니다
④	×	드라이빙/인너를 뒤바꿨습니다. NESTED LOOPS(Id 1)의 먼저 나온 자식은 Id 2 발권이므로 드라이빙은 발권이고, Id 4·5 항공편이 인너입니다. 발권_IX 가 (발권지점, 발권일자)라 항공편ID로는 탐색되지도 않습니다. 부분범위처리 이득도 드라이빙이 많을 때가 아니라 상위 일부만 필요할 때 커집니다

▪ 이 문제의 핵심

1. NESTED LOOPS의 먼저 나열된 자식이 드라이빙(선행) 집합입니다. 여기서는 발권(Id 2)이 드라이빙입니다.
2. 인너 접근은 드라이빙 건수만큼 반복됩니다. **항공편_IX** RANGE SCAN(Id 5)이 발권 620건마다 재실행됩니다.
3. 반복 횟수는 인너 크기가 아니라 드라이빙 건수가 결정합니다. 인너가 38만 건이어도 조인 반복은 620회입니다 — 병목을 인너 크기로 오귀속하지 않도록 주의합니다.
4. NL은 부분범위처리에 유리합니다. 드라이빙 앞쪽부터 조인해 내려가므로 상위 일부만 필요할 때 첫 행을 빠르게 냅니다.

- 한 줄 정리 NESTED LOOPS에서 위쪽 자식 발권이 드라이빙이고 620건마다 **항공편_IX** RANGE SCAN이 반복되며, 인너 반복 횟수는 인너 항공편 크기가 아니라 드라이빙 발권 건수가 결정합니다.

문제 11. 정답 ④

SELECT 절 스칼라 서브쿼리는 원칙적으로 바깥 쿼리의 행마다 한 번씩 실행됩니다. 이 반복 실행을 줄여 주는 장치가 스칼라 서브쿼리 캐싱입니다.

동작 : 바깥 행의 입력값을 해시 함수로 매핑 → 캐시(해시 테이블)에 결과 보관
 재호출 : 같은 입력값이 다시 오면 서브쿼리를 실제 실행하지 않고 캐시 값 반환
 한계 : 캐시 크기 유한 → 입력값 종류가 많으면 충돌·기존 항목 밀려남 → 적중률 저하

그래서 입력값의 종류가 적을수록(예: 소수의 코드값이 반복될수록) 캐시 적중이 잦아 실제 실행 횟수가 크게 줄고, 입력값 종류가 많을수록 캐시가 제 몫을 못 해 효과가 떨어집니다. ④은 이 해시 기반 캐시의 원리와 한계를 함께 정확히 짚었습니다. 나머지는 "캐싱"이라는 단어에서 곧바로 떠올리기 쉬운 반사적 연상을 결론으로 삼은 함정입니다.

①: 캐시는 세션 내내 유지되는 무한 저장소가 아니라 쿼리 수행 단위의 유한한 해시 캐시이며, 입력값을 키로 동작합니다. 캐시는 그 SQL의 수행이 끝나면 사라지므로 뒤이은 다른 SQL이 물려받지도 못하고, 입력값 종류가 여럿이면 호출도 한 번으로 줄지 않습니다.

②: 반대로 캐싱을 아예 없는 것으로 취급했습니다. 같은 입력값이 반복되면 캐시가 개입해 실제 호출이 줄므로 호출 횟수가 늘 바깥 행 수와 같아지지 않고, 조인으로 고쳐 쓰는 것만이 유일한 처방인 것도 아닙니다.

③: 스칼라 서브쿼리가 조인으로 변환(쿼리 변환)되는 경우가 있긴 하지만 항상 그런 것은 아닙니다. 변환 여부는 비용·형태에 따라 옵티마이저가 판단하며, 변환되지 않은 채 입력값 종류까지 많으면 캐시 적중률이 떨어져 실제로 성능이 나빠집니다.

선택지	판정	이유
①	×	캐시는 세션 내내 남는 무한 저장소가 아니라 해당 SQL 수행 단위로 살아 있는 유한한 해시 캐시이며, 키는 입력값입니다. 다음 SQL이 그 캐시를 물려받지 못하고, 입력값 종류가 여럿이면 호출도 한 번으로 줄지 않으므로 캐싱을 과장한 반사적 연상입니다
②	×	바깥 행마다 수행된다는 원칙은 맞지만 캐싱을 없는 것으로 취급했습니다. 같은 입력값이 반복되면 캐시가 개입해 실제 호출이 줄므로 호출 횟수가 늘 바깥 행 수와 같지 않고, 조인으로 고쳐 쓰는 것만이 유일한 처방인 것도 아닙니다
③	×	조인 변환이 일어나는 경우가 있다는 것까지는 맞지만 '항상'이 틀렸습니다. 변환 여부는 비용·형태에 따라 옵티마이저가 판단하며, 변환되지 않으면 캐싱을 동반한 스칼라 서브쿼리 방식 그대로 수행되므로 입력값 종류가 많을 때 성능이 나빠질 수 있습니다
④	○	입력값을 키로 한 해시 기반 캐시로 재호출을 대체해 실행 횟수를 줄이되, 캐시 크기 한계로 입력값 종류가 많으면 효과가 떨어진다는 원리·한계를 정확히 설명합니다

■ 이 문제의 핵심

1. **SELECT** 절 스칼라 서브쿼리는 원칙적으로 바깥 행마다 한 번씩 실행됩니다.
2. 스칼라 서브쿼리 캐싱은 입력값을 키로 한 해시 기반 캐시로, 같은 값의 재호출을 캐시로 대체해 실행 횟수를 줄입니다.
3. 캐시는 크기가 유한합니다 — 입력값 종류가 적으면 적중률이 높고, 많으면 충돌·밀려남으로 효과가 떨어집니다. 무한 캐시가 아닙니다.
4. 스칼라 서브쿼리의 조인 변환(쿼리 변환)은 조건부이며 항상 일어나는 것이 아닙니다.

■ 한 줄 정리 스칼라 서브쿼리 캐싱은 입력값을 키로 한 유한한 해시 캐시라 값 종류가 적을 때 실행 횟수를 크게 줄이며, 무한 캐시도 아니고 조인으로 늘 변환되는 것도 아닙니다.

문제 12. 정답 ②

CBO의 비용 산정에는 서로 독립인 두 축이 있습니다.

축 A) 카디널리티 추정 ← 통계(NDV·선택도·히스토그램)로 '몇 행이 나올지'를 예측

축 B) 액세스 경로 비용 ← 인덱스 경유 시 테이블 블록을 몇 번 방문하는지(클러스터링 팩터)

①④③은 모두 축 A를 정확히 설명합니다. 선택도는 조건을 만족하는 행의 비율 추정치이고 카디널리티는 여기에 전체 행 수를 곱해 얻으며(①), 등치 조건의 선택도는 대략 $1/NDV$ 라 NDV가 크면 카디널리티가 작아지고(④), 히스토그램은 편중된 분포에서 균일 분포 가정을 보정해 선택도를 정밀하게 만듭니다(③).

②는 이 축 A(카디널리티 통계)와 축 B(클러스터링 팩터)를 인과로 엮은 진술입니다. NDV는 값의 다양성을 나타내는 통계이고, 클러스터링 팩터는 인덱스 정렬 순서와 테이블 행의 물리적 저장 순서가 얼마나 일치하는가를 나타내는 별개의 값입니다. NDV가 큰 컬럼이라도 그 값들이 테이블에 흩어져 저장돼 있으면 클러스터링 팩터는 크고, 반대로 NDV가 작아도 같은 값끼리 모여 저장돼 있으면 작아집니다. 즉 NDV가 크다고 클러스터링 팩터가 작아지는 것이 아니며, 두 값 사이에 그런 함수 관계는 없습니다. ②는 없는 인과를 만든 별개 축 혼동이라 가장 부적절합니다.

선택지	판정	이유
①	○	선택도는 조건 만족 행의 비율 추정치이고, 예상 카디널리티는 전체 행 수에 선택도를 곱해 산정합니다 — 카디널리티 추정 축의 정의입니다
②	×	NDV(값의 다양성 통계)와 클러스터링 팩터(인덱스·테이블 정렬 일치도)는 독립 축입니다. NDV가 크다고 클러스터링 팩터가 작아지지 않으며, 둘을 인과로 이은 것은 별개 축 혼동입니다
③	○	편중된 분포의 컬럼에서 히스토그램은 균일 분포 가정을 보정해 특정 값의 선택도 추정 정밀도를 높입니다
④	○	등치 조건 선택도가 대략 $1/NDV$ 이므로 NDV가 클수록 선택도가 낮아져 예상 카디널리티가 작아집니다

■ 이 문제의 핵심

1. CBO 비용 산정은 '카디널리티 추정'과 '액세스 경로 비용'이라는 두 축으로 나뉩니다.

2. NDV·선택도·히스토그램은 카디널리티 추정 속입니다. 등치 선택도 $\approx 1/NDV$, 히스토그램은 편중 분포를 보정합니다.
3. 클러스터링 팩터는 별개 속입니다 — 인덱스 정렬 순서와 테이블 물리 저장 순서의 일치도로 결정되지, 값의 다양성(NDV)의 함수가 아닙니다.
4. "NDV가 크면 클러스터링 팩터가 작아진다"는 두 독립 속을 인과로 엮은 전형적인 별개 속 혼동입니다.
 - 한 줄 정리 NDV는 카디널리티를 추정하는 통계이고 클러스터링 팩터는 인덱스 액세스 비용을 좌우하는 별개의 물리적 정렬 지표이므로, "NDV가 크면 클러스터링 팩터도 작아진다"는 없는 인과를 만든 부적절한 진술입니다.

문제 13. 정답 ①

View Merging은 뷰 정의를 바깥 쿼리에 풀어 넣어 하나의 쿼리 블록으로 합치는 변환입니다. 다만 병합해도 결과 집합이 보존될 때만 안전하게 적용됩니다.

ROWNUM은 행에 수행 순서대로 매겨지는 값이라, 뷰를 바깥과 합쳐 조인 순서·필터가 바뀌면 어느 행에 어떤 번호가 매겨질지가 달라집니다. 즉 병합 시 결과가 보존되지 않으므로, 옵티마이저는 ROWNUM이 든 뷰를 병합하지 않고 별도 블록으로 수행합니다. ①이 이 원리를 정확히 진술합니다.

나머지는 키워드에서 오는 반사적 연상을 노린 함정입니다.

②: "GROUP BY = 집계 = 먼저 수행"이라는 인상으로 병합 불가를 단정하지만, 복합 View Merging은 GROUP BY·DISTINCT가 있는 뷰도 조건이 맞으면 병합합니다. GROUP BY의 존재만으로 병합에서 제외된다는 것은 반쪽 사실을 굳힌 오류입니다.

③: "병합 실패 = 최적화 끝"이라는 인상과 달리, 뷰가 병합되지 못해도 옵티마이저는 조건절 Pushing으로 바깥 조건을 뷰 안으로 밀어 넣어 뷰가 처리할 행을 줄일 수 있습니다. "밀어 넣지 못한다"는 성립하지 않습니다.

④: View Merging과 Unnesting을 "합친다"는 인상으로 한 묶음처럼 다뤘지만, 대상이 다른 독립 변환입니다. 뷰가 ROWNUM 때문에 병합되지 못하는 것과, 그 안의 서브쿼리가 Unnesting되는지는 서로 별개로 판단됩니다.

선택지	판정	이유
①	○	ROWNUM이 든 뷰는 병합 시 행별 번호가 달라져 결과가 보존되지 않으므로, 옵티마이저가 병합하지 않고 독립 블록으로 수행합니다
②	×	복합 View Merging은 GROUP BY·DISTINCT 뷰도 조건이 맞으면 병합합니다. "GROUP BY면 병합 제외"는 반쪽 사실을 전제로 굳힌 반사적 연상입니다
③	×	병합되지 못한 뷰에도 조건절 Pushing으로 바깥 조건을 밀어 넣어 처리 행을 줄일 수 있습니다. "밀어 넣지 못한다"는 사실이 아닙니다
④	×	View Merging과 Unnesting은 대상이 다른 독립 변환입니다. 뷰의 병합 불가가 그 안 서브쿼리의 Unnesting 여부를 결정하지 않습니다

▪ 이 문제의 핵심

1. View Merging은 결과가 보존될 때만 적용됩니다. ROWNUM은 병합 시 행별 번호가 달라져 결과를 바꾸므로 병합이 막힙니다.
2. 복합 View Merging은 GROUP BY·DISTINCT 뷰도 조건이 맞으면 병합합니다 — "집계 뷰는 병합 불가"는 반쪽 사실입니다.
3. 병합 실패가 최적화의 끝은 아닙니다. 조건절 Pushing으로 바깥 조건을 뷰 안에 밀어 넣어 처리량을 줄일 수 있습니다.
4. View Merging과 Unnesting은 독립 변환입니다. "둘 다 합치니 같이 막힌다"는 반사적 연상 오류입니다.

▪ 한 줄 정리 ROWNUM이 든 뷰는 병합 시 결과가 보존되지 않아 독립 블록으로 수행되며, GROUP BY 복합 뷰의 병합 가능성·no-merge 뷰로의 조건절 Pushing·Unnesting과의 독립성을 키워드 인상만으로 단정하는 것은 반사적 연상 오류입니다.

문제 14. 정답 ③

바인드 변수를 쓰면 SQL 텍스트가 동일해져 부모 커서(SQL_ID)를 공유하는 것은 맞습니다. 그러나 "텍스트가 같으니 언제나 하나의 실행계획만 재사용한다"는 것은 별개의 이야기입니다. v\$sql이 그 반례를 그대로 보여 줍니다.

같은 SQL_ID b7q2m9c4f0r3d 아래에 —

CHILD 0 : PLAN_HASH 1183726554, BIND_AWARE=N, 처리 12건 ← 소량(예: 'D')용 계획

CHILD 1 : PLAN_HASH 3902514877, BIND_AWARE=Y, 처리 480000건 ← 대량(예: 'A')용 계획

→ 텍스트는 같지만(부모 1개) 계획은 둘로 갈림(자식 2개)

승인상태처럼 히스토그램이 있고 값이 편중된 칼럼은 커서가 bind sensitive가 되고, 바인드 값에 따라 처리건수가 크게 달라지면 오라클이 적응적 커서 공유(Adaptive Cursor Sharing)로 bind aware 자식 커서를 하나 더 만듭니다. 요약표의 CHILD 0(12건)과 CHILD 1(480,000건)은 PLAN_HASH_VALUE가 서로 다른 별개 계획입니다. 그러므로 "바인드 변수를 쓰면 값과 무관하게 항상 자식 커서 0번 하나만 재사용한다"는 ③는 실제 동작과 어긋난 부적절한 진술입니다. 바인드 변수 사용이 곧 단일 계획을 보장한다는 정규화 가정의 오류입니다.

선택지	판정	이유
①	○	같은 SQL_ID(b7q2...) 아래 CHILD 0·1이 서로 다른 PLAN_HASH_VALUE를 가지므로, 부모 하나에 실행계획이 다른 자식 커서 둘이 맞습니다
②	○	히스토그램이 있어 IS_BIND_SENSITIVE=Y가 되고, 처리건수 차이(12 vs 480000)를 감지해 IS_BIND_AWARE=Y 자식이 생성됩니다 — 표와 일치합니다
③	×	텍스트가 같아도 편중 칼럼에서는 ACS가 bind aware 자식(1번)을 추가로 만듭니다. v\$sql의 두 자식·두 PLAN_HASH가 "항상 0번 하나만 재사용"을 반증합니다
④	○	첫 실행의 bind peeking 값이 계획을 좌우하는 것은 편중 칼럼의 잘 알려진 위험이며, ACS가 이를 보완하려는 장치입니다

■ 이 문제의 핵심

1. 바인드 변수는 텍스트를 같게 해 부모 커서(SQL_ID)를 공유시키지만, 그것이 단일 실행계획을 보장하지는 않습니다.
2. 히스토그램 + 값 편중 → bind sensitive → 적응적 커서 공유(ACS)로 bind aware 자식 커서가 걸립니다.
3. 같은 SQL_ID라도 CHILD_NUMBER·PLAN_HASH_VALUE가 다르면 별개 계획입니다. 자식 커서 개수가 곧 걸린 계획 수입니다.
4. bind peeking은 첫 실행 값에 계획을 고정하므로, 편중 칼럼에서 처음 엿본 값이 이후 성능을 좌우할 수 있습니다 — ACS가 이를 보완합니다.

■ 한 줄 정리 바인드 변수로 텍스트는 같아져 부모 커서를 공유하지만, 편중 칼럼에서는 적응적 커서 공유가 계획이 다른 bind aware 자식 커서를 만들어 "값과 무관하게 항상 한 계획만 재사용"은 성립하지 않습니다.

문제 15. 정답 ③

직접 경로 삽입(/**+ APPEND */)은 대상 세그먼트에 대해 배타적 TM 락(exclusive)을 획득합니다. 그래서 적재가 커밋(또는 롤백)되기 전까지 같은 테이블에 대한 다른 세션의 일반 INSERT/UPDATE/DELETE는 차단됩니다. ③은 이 동작을 정확히 뒤집어 "배타적 잠금을 걸지 않아 동시 INSERT가 가능하다"고 진술한 정반대 진술입니다.

INSERT /**+ APPEND */ 수행 중 -

- 대상 테이블 운송장_집계 : 배타적 TM 락 보유
- 다른 세션의 일반 INSERT : 커밋될 때까지 대기(차단) ← ③은 이를 "동시 가능"으로 뒤집음
- 적재 방식 : HWM 위 새 블록에만 기록(프리 블록 재사용 안 함)

나머지 ①②④는 직접 경로 삽입·NOLOGGING의 실제 동작을 옳게 서술합니다. ①은 HWM 위 적재와 프리 블록 미재사용, ②는 NOLOGGING+직접경로의 redo 최소화과 미디어 복구 불가·백업 필요, ④는 데이터 redo만 줄고 인덱스 유지 비용·redo는 남는다는 점입니다. 부적절한 것은 잠금 동작을 반대로 말한 ③입니다.

선택지	판정	이유
①	○	APPEND 직접 경로는 HWM 위 새 블록에 적재하고 세그먼트의 기존 프리 블록·빈 공간을 재사용하지 않습니다 — 정확합니다
②	○	NOLOGGING+직접경로는 데이터 redo를 최소화하지만 그 구간은 미디어 복구가 불가하므로, 적재 직후 백업이 필요하다는 것은 옳은 권고입니다
③	×	직접 경로 삽입은 대상 테이블에 배타적 락을 걸어 커밋 전까지 타 세션의 일반 DML을 차단합니다. "동시 INSERT 가능"은 동작을 정반대로 뒤집은 진술입니다
④	○	NOLOGGING은 데이터 블록 적재의 redo만 줄일 뿐, 운송장_집계_IX 인덱스 유지와 그 redo는 그대로 발생합니다 — 정확합니다

■ 이 문제의 핵심

1. /**+ APPEND */ 직접 경로 삽입은 대상 테이블에 배타적 락**을 걸어, 커밋 전까지 다른 세션의 일반 DML을 차단합니다.
2. 직접 경로는 HWM 위 새 블록에만 적재하고 세그먼트 안 프리 블록·빈 공간을 재사용하지 않습니다.
3. NOLOGGING+직접경로는 데이터 redo만 최소화하며, 그 구간은 미디어 복구가 불가하므로 적재 직후 백업이 필요합니다.

4. NOLOGGING이라도 인덱스 유지 비용과 인덱스 redo는 남습니다 — 데이터 블록의 redo만 줄어듭니다.

- 한 줄 정리 직접 경로 삽입은 대상 테이블에 배타적 락을 걸어 커밋 전 동시 DML을 차단하므로, "배타적 잠금을 걸지 않아 동시 INSERT가 가능하다"는 ③은 동작을 정반대로 뒤집은 진술입니다.

문제 16. 정답 ①

Partition Pruning은 옵티마이저가 WHERE 조건의 값을 파티션 경계(VALUES LESS THAN)와 대조해 스캔 대상 파티션을 미리 좁히는 최적화입니다. 이 대조가 가능하려면 조건이 파티션 키 칼럼(청구연월) 자체에 가공 없이 걸려 있어야 합니다.

- ④ 청구연월 = 202603 → pt_202603 한 개
- ② 청구연월 ∈ [202601, 202602] → pt_202601, pt_202602 두 개
- ③ 청구연월 >= 202605 → pt_202605, pt_202606, pt_max
- ① TO_CHAR(청구연월) = '202603' → 키에 함수, 경계 대조 불가 → 전 파티션 스캔

④②③은 청구연월을 가공하지 않은 정수 값·범위로 걸었으므로, 옵티마이저가 경계와 대조해 해당 파티션만 접근합니다(②·③처럼 여러 파티션에 걸쳐도 나머지는 건너뛰므로 Pruning은 유효합니다).

①은 TO_CHAR(청구연월)로 파티션 키 숫자에 형변환 함수를 씌워 문자열과 비교합니다. 그러면 옵티마이저는 TO_CHAR 결과값을 파티션 경계(숫자)와 대조할 수 없어 Pruning을 포기하고, pt_202601 ... pt_max 전 파티션을 스캔한 뒤 각 행에서 TO_CHAR를 평가해 필터합니다. 조건이 의미상 "2026년 3월"을 겨냥하는 부분적 진실이라도, 형변환 한 줄이 최선(1개 파티션)을 최악(전 파티션 스캔)으로 뒤집습니다. 그래서 Pruning이 효과적으로 작동한다고 볼 수 없는 것은 ①입니다.

선택지	판정	이유
①	×	파티션 키 숫자에 TO_CHAR를 씌워 경계 대조가 불가능해집니다. 전 파티션을 스캔한 뒤 필터하므로 Pruning이 깨집니다
②	○	BETWEEN 구간 [202601, 202602]가 경계와 대조돼 pt_202601·pt_202602 두 파티션만 접근합니다
③	○	>= 202605 하한이 경계와 대조돼 pt_202605·pt_202606·pt_max만 읽고 앞선 파티션은 건너뛸니다. Pruning 작동입니다
④	○	청구연월을 가공 없이 = 202603으로 걸어 pt_202603 한 개로 정확히 Pruning됩니다

▪ 이 문제의 핵심

1. Partition Pruning은 파티션 키 칼럼에 가공 없이 걸린 조건에서만 작동합니다. 키에 함수를 씌우면 경계와 대조할 수 없습니다.
2. 정수 YYYYMM 키의 등치·범위 조건(=, BETWEEN, >=)은 경계와 그대로 대조되어 필요한 파티션만 남깁니다.
3. 여러 파티션에 걸치더라도(②·③) Pruning은 유효합니다 — 나머지 파티션을 건너뛰기만 하면 됩니다.
4. TO_CHAR처럼 숫자 키를 문자로 형변환하면 Pruning이 무력화돼 전 파티션 스캔을 유발합니다. "월만 뽑았다"는 부분적 타당함이 함정입니다.

- 한 줄 정리 정수 YYYYMM 키를 가공하지 않은 값·범위는 Pruning이 작동하지만, TO_CHAR(청구연월)처럼 키에 형변환을 씌우면 경계 대조가 불가능해져 전 파티션을 스캔합니다.

문제 17. 정답 ③

분배 방식은 조인 노드(Id 8)의 두 입력이 PX SEND를 거치는지, 그리고 조인을 감싸는 노드가 무엇인지로 읽습니다.

- Id 7 PX PARTITION HASH ALL (Pstart 1~Pstop 32) 조인을 파티션 단위로 감쌌
- Id 8 HASH JOIN (Q1,00) 파티션 쌍 내부에서 조인
- Id 9 TABLE ACCESS FULL 펀드거래 (Q1,00, 1~32) 조인 입력 - PX SEND 없음
- Id 10 TABLE ACCESS FULL 펀드잔고 (Q1,00, 1~32) 조인 입력 - PX SEND 없음

두 조인 입력(Id 9·10)은 같은 TQ(Q1,00) 아래에서, 같은 PX PARTITION HASH ALL(Id 7) 안에서 읽힙니다. 그 사이 어디에도 PX SEND/PX RECEIVE가 없습니다. 두 테이블이 투자자ID로 동일하게 32개 해시 파티셔닝돼 있으므로, 각 병렬 서버가 대응하는 파티션 쌍(펀드거래 p_k ↔ 펀드잔고 p_k)만 맞들어 조인합니다. 이것이 재분배가 사라지는 (full) 파티션 와이즈 조인입니다.

플랜에 유일하게 있는 PX SEND HASH(Id 5)는 조인 위쪽입니다. 조인·부분집계(Id 6 HASH GROUP BY)를 마친 결과를 펀드유형으로 다시 해시 재분배해 최종 HASH GROUP BY(Id 3)로 넘기는, 집계용 재분배입니다. 조인 키(투자자ID) 재분배가 아닙니다.

- 조인 분배(Id 7~10) : 재분배 없음 - 동일 파티셔닝으로 파티션 쌍 로컬 조인 → 파티션 와이즈
- Id 5 PX SEND HASH : 펀드유형 집계용 재분배 (조인 아님)

Id 4 PX RECEIVE : Id 5가 보낸 부분집계 결과를 받는 쪽 (조인 입력 아님)
 Id 7 Pstart 1~32 : 조인 대상 파티션 번호 범위 (버킷 범위 · 복제 범위 아님)

조인 입력에 SEND가 없다는 사실이 hash-hash·broadcast 계열을 모두 배제합니다. 따라서 ③이 옳습니다.

나머지 셋은 노드와 분배 방식을 뒤바꾼 값스왑입니다.

- ② hash-hash : 조인 입력(Id 9·10)에 PX SEND HASH가 있어야 성립 → 없음
 Id 5·4는 조인 위(Id 6 HASH GROUP BY 다음) 집계용 TQ
- ① broadcast-none: PX SEND BROADCAST 노드가 어디에도 없음
 Id 5는 SEND HASH지 SEND BROADCAST가 아니므로 복제 경로가 될 수 없음
- ④ hash-broadcast: PX SEND HASH·PX SEND BROADCAST 둘 다 조인 입력에 없음
 Id 7의 1~32는 PX PARTITION HASH ALL이 훑는 파티션 번호지 복제 범위가 아님

선택지	판정	이유
①	×	broadcast-none이면 소형 쪽에 PX SEND BROADCAST 가 있어야 하는데 플랜에 BROADCAST 노드가 없습니다. Id 5는 SEND HASH라 복제 경로가 될 수 없고, 그마저 조인 위(Id 6 다음)에 있습니다. Id 9·10 모두 Pstart 1~Pstop 32의 파티션 단위 로컬 스캔입니다
②	×	hash-hash면 조인 입력 양쪽에 PX SEND HASH 가 있어야 합니다. Id 9·10엔 SEND가 없고, Id 5 SEND HASH와 Id 4 PX RECEIVE는 Id 6 HASH GROUP BY 위쪽 TQ라 집계용입니다. Id 7의 1~32도 버킷 범위가 아니라 훑는 파티션 번호입니다
③	○	조인 입력 Id 9·10이 같은 Q1,00·같은 PX PARTITION HASH ALL(Id 7) 아래에서 PX SEND 없이 읽혀 파티션 쌍만 조인합니다. 동일 파티셔닝이 재분배를 얹은 파티션 와이즈 조인이며, 유일한 SEND(Id 5)는 조인 위 펀드유형 HASH GROUP BY 재분배입니다
④	×	hash-broadcast면 한쪽 PX SEND HASH +다른 쪽 PX SEND BROADCAST 가 조인 입력에 있어야 합니다. Id 9·10엔 어느 SEND도 없고 플랜에 BROADCAST 노드 자체가 없습니다. Id 7의 Pstart 1~Pstop 32는 복제 범위가 아니라 조인이 훑는 파티션 번호입니다

▪ 이 문제의 핵심

- 조인 분배는 조인 입력이 PX SEND를 거치는지로 판별합니다. 두 입력(Id 9·10)에 SEND가 없고 같은 PX PARTITION HASH ALL(Id 7) 아래 있으면 파티션 와이즈 조인입니다.
- 파티션 와이즈 조인의 조건은 양쪽이 조인 키로 동일 파티셔닝된 것입니다. 각 서버가 대응 파티션 쌍만 로컬 조인해 재분배가 사라집니다.
- 플랜 속 PX SEND HASH가 곧 조인 재분배는 아닙니다. Id 5의 SEND HASH는 조인 위 펀드유형별 HASH GROUP BY를 위한 집계용 재분배입니다 — 위치(조인 아래 vs 위)를 확인해야 합니다.
- hash-hash·broadcast-none·hash-broadcast는 조인 입력에 **PX SEND HASH/PX SEND BROADCAST**가 있어야 성립합니다. 이 플랜의 조인 입력엔 어느 SEND도 없습니다.

▪ 한 줄 정리 조인 입력 펀드거래·펀드잔고가 같은 Q1,00·같은 PX PARTITION HASH ALL 아래에서 PX SEND 없이 대응 파티션 쌍만 조인하므로 파티션 와이즈 조인이며, Id 5 PX SEND HASH는 조인이 아니라 펀드유형 집계용 재분배입니다.

문제 18. 정답 ②

UNION과 UNION ALL의 결정적 차이는 중복 제거이고, 그 중복 제거가 정렬 단계의 유무를 가릅니다.

UNION은 두 집합을 합친 뒤 중복을 제거하며, 이를 위해 내부적으로 Sort Unique 또는 해시 단계를 거칩니다.

UNION ALL은 중복을 제거하지 않고 그대로 이어 붙이므로 그 정렬 단계가 없습니다.

②가 이 차이를 정확히 진술합니다.

나머지는 일부 항목의 성질을 전체로 부풀린 하위항목 전체화 오류입니다.

- ①: 윈도우 함수에 **ORDER BY**가 없어도 **PARTITION BY**가 있으면 행을 파티션 키로 나눠야 하므로, 실행계획에는 대개 이를 위한 정렬(WINDOW SORT)이나 그룹핑이 나타납니다. "정렬 없이 입력 순서대로"는 사실이 아닙니다.
- ③: "집합 연산자는 정렬로 중복을 제거한다"는 성질은 **UNION·INTERSECT·MINUS**에 해당하는 부분 사실입니다. 이를 **UNION ALL**까지 확장한 것이 오류입니다 — **UNION ALL**은 중복을 제거하지 않아 그 정렬 단계 자체가 없습니다.
- ④: **INTERSECT**도 공통 행을 판별하며 중복을 제거하므로 각 입력에 Sort Unique가 붙습니다. 정렬을 통한 중복 제거를 **UNION**에만 있는 성질로 좁힌 것 역시 부분을 전체로 오귀속한 것입니다.

선택지	판정	이유
①	×	PARTITION BY 가 있으면 파티션 키로 행을 나눠야 해 대개 WINDOW SORT가 나타납니다. "정렬 없이 입력 순서대로"는 성립하지 않습니다
②	○	UNION 은 중복 제거를 위해 Sort Unique/해시를 거치고, UNION ALL 은 중복 제거가 없어 그 정렬 단계 없이 이어 붙입니다
③	×	"정렬로 중복 제거"는 UNION · INTERSECT · MINUS 의 부분 성질입니다. 중복을 제거하지 않는 UNION ALL 까지 확장한 하위항목 전체화 오류입니다
④	×	INTERSECT 도 중복을 제거하며 각 입력에 Sort Unique가 붙습니다. 정렬 중복 제거를 UNION 만의 성질로 좁힌 오귀속입니다

▪ 이 문제의 핵심

1. UNION은 중복 제거(Sort Unique/해시) 단계가 든다. UNION ALL은 이어 붙일 뿐이라 그 단계가 없습니다.
2. 정렬을 통한 중복 제거는 UNION · INTERSECT · MINUS의 공통 성질입니다 — UNION만의 것이 아닙니다.
3. UNION ALL은 집합 연산자이지만 중복을 제거하지 않습니다. "집합 연산자니까 정렬로 중복 제거"를 UNION ALL에 적용하면 틀립니다.
4. PARTITION BY가 있으면 ORDER BY가 없어도 파티션 분할을 위한 정렬(WINDOW SORT)이 대개 필요합니다.

▪ 한 줄 정리 UNION은 중복 제거를 위해 Sort Unique를 거치고 UNION ALL은 그 단계 없이 이어 붙이며, "정렬 중복 제거"를 UNION ALL에 확장하거나 UNION에만 가두는 것은 부분 성질을 전체로 부풀린 하위항목 전체화 오류입니다.

문제 19. 정답 ④

SQL Server의 읽기 일관성은 어느 시점의 데이터를 보느냐로 같습니다. 핵심은 스냅샷의 범위가 '문장 단위'냐 '트랜잭션 단위'냐입니다.

격리수준	읽는 기준 시점	같은 행 재조회 시
READ COMMITTED(기본)	문장 실행 시점(락 기반)	값이 바뀔 수 있음 → NRR 가능
RCSI	문장 시작 시점(버전)	문장마다 갱신 → NRR 가능
SNAPSHOT	트랜잭션 시작 시점(버전)	트랜잭션 내내 고정 → NRR 방지

SNAPSHOT 격리수준은 트랜잭션이 시작한 시점의 버전을 트랜잭션이 끝날 때까지 일관되게 읽습니다(버전 기반 MVCC). 그래서 트랜잭션 안에서 같은 행을 몇 번 읽어도 그 사이 다른 트랜잭션이 값을 바꿔 커밋했든 아니든 같은 값을 봅니다. Non-Repeatable Read가 방지됩니다. ④이 정확합니다.

- ①: 기본 READ COMMITTED의 공유(S) 락은 읽은 직후 바로 해제됩니다(트랜잭션 끝까지 유지하지 않음). 그래서 같은 행을 다시 읽는 사이 다른 트랜잭션이 값을 바꿔 커밋하면 값이 달라져 Non-Repeatable Read가 가능합니다. "S 락을 끝까지 유지"는 사실과 반대입니다.
- ②: RCSI는 스냅샷을 문장(statement) 단위로 잡습니다. 한 트랜잭션 안이라도 문장이 새로 시작하면 그 시점의 최신 커밋 값을 읽으므로, 두 문장이 같은 행을 읽으면 값이 달라질 수 있어 Non-Repeatable Read가 여전히 가능합니다.
- ③: "커밋된 데이터만 읽는다"는 성질은 RC · RCSI · SNAPSHOT에 공통이라 판별 근거가 못 됩니다(공통단서 무력화). RCSI는 문장 단위, SNAPSHOT은 트랜잭션 단위 스냅샷이라 Non-Repeatable Read 방지 수준이 다릅니다.

선택지	판정	이유
①	×	기본 READ COMMITTED의 S 락은 읽은 직후 해제되지 트랜잭션 끝까지 유지되지 않습니다. 재조회 시 값이 바뀔 수 있어 Non-Repeatable Read가 가능합니다
②	×	RCSI 스냅샷은 문장 단위라 한 트랜잭션 안에서도 문장이 바뀌면 최신 커밋 값을 읽습니다. Non-Repeatable Read가 여전히 가능합니다
③	×	"커밋된 데이터만 읽음"은 RC · RCSI · SNAPSHOT 공통 성질이라 판별 근거가 못 됩니다. 문장 단위(RCSI)와 트랜잭션 단위(SNAPSHOT)의 방지 수준은 다릅니다
④	○	SNAPSHOT은 트랜잭션 시작 시점 버전을 끝까지 읽으므로 같은 행 재조회 시 값이 고정되어 Non-Repeatable Read가 방지됩니다

▪ 이 문제의 핵심

1. 읽기 일관성의 관건은 스냅샷의 범위 — 문장 단위냐 트랜잭션 단위냐가 Non-Repeatable Read 방지 여부를 가릅니다.
2. 기본 READ COMMITTED(락 기반)의 S 락은 읽은 직후 해제됩니다. 재조회 값이 바뀔 수 있어 Non-Repeatable Read가 가능합니다.

3. RCSI는 문장 단위 스냅샷이라 한 트랜잭션 안 두 문장이 다른 값을 볼 수 있습니다 — Non-Repeatable Read를 막지 못합니다.

4. SNAPSHOT은 트랜잭션 단위 스냅샷(버전 기반 MVCC)이라 트랜잭션 내내 값이 고정되어 Non-Repeatable Read를 방지합니다.

▪ 한 줄 정리 SNAPSHOT은 트랜잭션 시작 버전을 끝까지 읽어 Non-Repeatable Read를 방지하지만, 문장 단위인 RCSI와 S 락을 즉시 푸는 기본 READ COMMITTED는 막지 못하며, "커밋된 값만 읽음"은 세 격리수준 공통이라 판별 근거가 되지 못합니다.

문제 20. 정답 ①

블로킹은 같은 자원(여기서는 object_id 154227의 보험청구 테이블)에 한쪽은 락을 보유(LMODE)하고 다른 쪽은 요청(REQUEST)만 걸린 채 대기할 때 발생합니다. 표를 두 행으로 갈라 읽으면 방향과 호환성이 한 곳에서 정해집니다.

SID 44 : TM LMODE=4(S) REQUEST=0 BLOCK=1 → S 락 보유, 남을 막는 중(블로커)
 SID 61 : TM LMODE=0 REQUEST=3(RX) → RX 요청, 아직 못 얻음(대기·피해자)

호환성 : S(4) ↔ RX(3) = 비호환 → 61은 44가 S를 풀 때까지 대기
 BLOCK : =1은 '보유한' 44에 붙는다(44가 막는 쪽)

①는 두 가지를 모두 뒤집었습니다. 첫째, S(4)와 RX(3)를 "호환된다"고 했지만 자극물 각주와 실제 TM 호환 규칙상 비호환이라 61은 즉시 획득할 수 없습니다. 둘째, **BLOCK=1**은 락을 보유해 남을 막는 세션(44)에 붙는 값인데, ①는 이를 "61이 44를 막는다"로 방향까지 반대로 해석했습니다. 그래서 부적절한 진술은 ①입니다. 반면 ④②③은 블로커(44)·피해자(61)·비호환에 따른 대기를 옳게 서술합니다.

선택지	판정	이유
①	×	S(4)와 RX(3)는 비호환이라 61은 즉시 획득할 수 없고, BLOCK=1은 보유 세션 44에 붙어 44가 막는 쪽입니다. 호환성과 블로킹 방향을 모두 반대로 진술했습니다
②	○	SID 61은 REQUEST=3(RX), LMODE=0으로 아직 락을 얻지 못한 대기 상태입니다 — 표의 값 그대로입니다
③	○	S(4)와 RX(3)는 비호환이므로 61의 RX 요청은 44가 S를 해제할 때까지 대기합니다. 방향·근거가 정확합니다
④	○	SID 44는 LMODE=4(S)로 락을 보유하고 BLOCK=1이 붙어 있어, 다른 세션을 막는 블로커가 맞습니다

▪ 이 문제의 핵심

1. 블로킹은 같은 자원에서 LMODE(보유) ↔ REQUEST(요청)가 마주칠 때 발생하며, 요청만 걸린 쪽이 대기·피해자입니다.
2. **BLOCK=1**은 락을 보유해 남을 막는 세션에 붙습니다 — 대기하는 세션이 아니라 블로커 쪽 표시입니다.
3. TM 모드 S(4)와 RX(3)는 비호환입니다. **LOCK TABLE ... IN SHARE MODE**가 걸린 테이블에는 UPDATE의 RX 요청이 대기합니다.
4. 방향은 값으로 확정합니다: LMODE가 채워진 44가 블로커, REQUEST만 걸린 61이 피해자입니다.

▪ 한 줄 정리 SID 44가 S(4) 락을 보유(BLOCK=1)해 블로커이고, 비호환인 RX(3)를 요청한 SID 61이 대기·피해자이므로, 호환성과 블로킹 방향을 모두 뒤집은 ①가 부적절합니다.